

## ABSTRACT

Despite the huge advancement made in modern communication technology, there exists a huge communication gap between the general public and the people with speech and hearing impairments. This barrier tends to lead to social exclusion and limited access to normal interaction in education, health care, work and public services. Many sign language translation systems have been proposed based on Artificial Intelligence (AI), but many of these proposals have disadvantages such as lack of datasets, poor real-time performance and inefficient processing techniques. Furthermore, it is very difficult to implement in practice because there is no software for regional sign languages, especially PSL (Pakistani Sign Language).

This project is an Intelligent Sign Language Recognition and Translation System Using Artificial Intelligence, Computer Vision and Machine Learning. The system is built to recognize static gestures of Pakistani Sign Language from live camera feed. It uses MediaPipe for state-of-the-art hand detection, 21 hand landmark coordinates for feature extraction and a small Multilayer Perceptron (MLP) neural network for classification. Microsoft Edge Neural Text to Speech (TTS) provides useful text output and synthesized Urdu speech for recognized gestures.

Compared to available implementations, the solution we present overcomes the gaps by providing: 1) a dense and well-organized PSL dataset annotated with PSL descriptions; 2) pre-processing methods which are well-tuned like data-augmentation, noise removal and normalization; and finally 3) full stack client-server web application. We attain around 98.28% accuracy on our test data of 37 PSL alphabet classes while maintaining real-time performance of around 20 frames per second. Our project pushes forward accessible technology that considers Pakistan's deaf and speech impaired by focusing on criteria such as performance, usability, and local language integration.

# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>LIST OF FIGURES .....</b>                                           | <b>1</b>  |
| <b>LIST OF TABLES .....</b>                                            | <b>2</b>  |
| <b>LIST OF ABBREVIATIONS.....</b>                                      | <b>3</b>  |
| <b>CHAPTER 1 .....</b>                                                 | <b>5</b>  |
| <b>INTRODUCTION .....</b>                                              | <b>5</b>  |
| <b>1.1 INTRODUCTION.....</b>                                           | <b>6</b>  |
| <b>1.2 Background .....</b>                                            | <b>7</b>  |
| <b>1.3 Problem Statement .....</b>                                     | <b>8</b>  |
| <b>1.4 Objectives.....</b>                                             | <b>8</b>  |
| <b>1.5 Scope of the Project.....</b>                                   | <b>9</b>  |
| <b>CHAPTER 2 .....</b>                                                 | <b>10</b> |
| <b>LITERATURE REVIEW .....</b>                                         | <b>10</b> |
| <b>2.1 LITERATURE REVIEW .....</b>                                     | <b>11</b> |
| <b>2.3 International Sign Language Recognition Research .....</b>      | <b>12</b> |
| <b>2.4 Pakistan Sign Language Recognition Research.....</b>            | <b>12</b> |
| <b>2.5 Deep Learning Techniques in Sign Language Recognition .....</b> | <b>13</b> |
| <b>2.6 Comparative Analysis and Research Gap.....</b>                  | <b>13</b> |
| <b>CHAPTER 3 .....</b>                                                 | <b>15</b> |
| <b>SYSTEM REQUIREMENTS SPECIFICATION .....</b>                         | <b>15</b> |
| <b>3.1 SYSTEM REQUIREMENTS SPECIFICATION.....</b>                      | <b>16</b> |
| <b>3.2 Overall System Description .....</b>                            | <b>16</b> |
| <b>3.3 Functional Requirements.....</b>                                | <b>16</b> |
| <b>3.3.1 User Requirements.....</b>                                    | <b>16</b> |
| <b>3.3.2 System Requirements .....</b>                                 | <b>17</b> |
| <b>3.3.3 Data Processing Requirements .....</b>                        | <b>17</b> |
| <b>3.3.4 Gesture Recognition Requirements.....</b>                     | <b>17</b> |
| <b>3.3.5 Dashboard and Progress Tracking.....</b>                      | <b>18</b> |
| <b>3.3.6 Settings and User Preferences .....</b>                       | <b>18</b> |
| <b>3.4 Non-Functional Requirements.....</b>                            | <b>18</b> |

|                                                   |    |
|---------------------------------------------------|----|
| 3.4.1 Performance .....                           | 18 |
| 3.4.2 Accuracy .....                              | 19 |
| 3.4.3 Reliability .....                           | 19 |
| 3.4.4 Usability .....                             | 19 |
| 3.4.5 Scalability .....                           | 19 |
| 3.4.6 Security and Privacy .....                  | 20 |
| 3.5 Hardware Requirements.....                    | 20 |
| 3.6 Software Requirements.....                    | 20 |
| CHAPTER 4 .....                                   | 22 |
| SYSTEM DESIGN AND MODELING .....                  | 22 |
| 4.1 SYSTEM DESIGN AND MODELING .....              | 23 |
| 4.2 Architectural Design.....                     | 23 |
| 4.3 Design Models.....                            | 25 |
| 4.3.1 Use Case Diagram .....                      | 25 |
| 4.3.2 Activity Diagram .....                      | 26 |
| 4.3.3 Class Diagram.....                          | 27 |
| 4.3.4 Data Flow Diagram-Level 0 .....             | 28 |
| 4.3.5 Data Flow Diagram-Level 1 .....             | 28 |
| 4.3.6 System Sequence Diagram .....               | 29 |
| CHAPTER 5 .....                                   | 30 |
| PROPOSED METHODOLOGY.....                         | 30 |
| 5.1 PROPOSED METHODOLOGY .....                    | 31 |
| 5.2 Overall Methodology Overview .....            | 31 |
| 5.3 Data Collection and Dataset Construction..... | 31 |
| 5.4 Data Preprocessing.....                       | 33 |
| 5.5 Feature Extraction .....                      | 34 |
| 5.6 Model Development and Training.....           | 35 |
| 5.6.1 Alphabet Recognition Model .....            | 35 |
| 5.6.2 Word Recognition Model .....                | 36 |
| 5.6.3 Word Recognition Model .....                | 37 |
| 5.7 Sign to Text Conversion .....                 | 37 |
| 5.8 System Integration .....                      | 38 |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>CHAPTER 6 .....</b>                                            | <b>39</b> |
| <b>IMPLEMENTATION AND RESULTS .....</b>                           | <b>39</b> |
| <b>6.1 IMPLEMENTATION AND RESULTS .....</b>                       | <b>40</b> |
| <b>6.2 Implementation Overview .....</b>                          | <b>40</b> |
| <b>6.3 Technology Stack .....</b>                                 | <b>40</b> |
| <b>6.4 Backend Implementation .....</b>                           | <b>40</b> |
| <b>6.5 Frontend Implementation .....</b>                          | <b>41</b> |
| <b>6.6 Rest Api Endpoint .....</b>                                | <b>41</b> |
| <b>6.7 Real Time Data Pipeline .....</b>                          | <b>42</b> |
| <b>6.8 Model Training and Evaluation .....</b>                    | <b>43</b> |
| <b>6.8.1 Dataset Preparation .....</b>                            | <b>43</b> |
| <b>6.8.2 Training Configuration .....</b>                         | <b>43</b> |
| <b>6.8.3 Training and Testing Accuracy Results .....</b>          | <b>43</b> |
| <b>6.8.4 Confusion Matrix Analysis .....</b>                      | <b>44</b> |
| <b>6.8.5 Performance Metrics Summary .....</b>                    | <b>45</b> |
| <b>6.9 System Interface and Output Design .....</b>               | <b>45</b> |
| <b>6.9.1 Landing Page .....</b>                                   | <b>45</b> |
| <b>6.9.2 Sign Detection Page .....</b>                            | <b>46</b> |
| <b>6.9.3 Learn Sign Page .....</b>                                | <b>47</b> |
| <b>6.9.4 Dictionary Page .....</b>                                | <b>48</b> |
| <b>6.9.5 Dashboard Page .....</b>                                 | <b>49</b> |
| <b>6.9.6 Setting Page .....</b>                                   | <b>50</b> |
| <b>6.10 Advanced System Features .....</b>                        | <b>50</b> |
| <b>6.10.1 Sentence Builder .....</b>                              | <b>50</b> |
| <b>6.10.2 Dual-Mode Recognition .....</b>                         | <b>50</b> |
| <b>6.10.3 Urdu Text to Speech .....</b>                           | <b>51</b> |
| <b>6.10.4 Real Time MJPEG Video Streaming .....</b>               | <b>51</b> |
| <b>6.10.5 Configurable Backend and Re Training Pipeline .....</b> | <b>51</b> |
| <b>6.10.6 Learning Analytics Dashboard .....</b>                  | <b>51</b> |
| <b>CHAPTER 7 .....</b>                                            | <b>52</b> |
| <b>CONCLUSION AND FUTURE WORK .....</b>                           | <b>52</b> |
| <b>7.1 CONCLUSION .....</b>                                       | <b>53</b> |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>7.2 Future Work.....</b>                                      | <b>54</b> |
| <b>7.2.1 Production Deployment .....</b>                         | <b>54</b> |
| <b>7.2.2 Dynamic Gesture and Sentence-Level recognition.....</b> | <b>54</b> |
| <b>7.2.3 Dataset Expansion.....</b>                              | <b>54</b> |
| <b>7.2.4 Intelligent Analytics and Adaptive Learning.....</b>    | <b>55</b> |
| <b>7.2.5 Domain Expansion .....</b>                              | <b>55</b> |
| <b>REFERENCES.....</b>                                           | <b>56</b> |

## LIST OF FIGURES

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Figure 4.2: Lingua Sign System Architecture Diagram.....        | 24 |
| Figure 4.3.2: Activity diagram of Lingua Sign System .....      | 26 |
| Figure 4.3.3: Class Diagram of Lingua Sign System.....          | 27 |
| Figure 4.3.5 shows the Data Flow Diagram of Level 1. ....       | 29 |
| Figure 4.3.6: System Sequence Diagram .....                     | 29 |
| Figure 5.3: Dataset Collection and construction.....            | 32 |
| Figure 5.5: Feature Extraction .....                            | 34 |
| Figure 6.8.3 shows the training and testing accuracy .....      | 44 |
| Figure 6.8.4 shows the confusion matrix for alphabet model..... | 44 |
| Figure 6.9.1: Landing Page.....                                 | 46 |
| Figure 6.9.2 Detecting page .....                               | 47 |
| Figure 6.9.3 Sign Learn Page.....                               | 48 |
| Figure 6.9.4 : The Dictionary Page. ....                        | 49 |
| Figure 6.9.5 : Dash Board Page. ....                            | 49 |
| Figure 6.9.6 : Setting Page.....                                | 50 |

## LIST OF TABLES

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Table 2.6: Comparative Summary of Related Sign Language Recognition Systems.....     | 14 |
| Table 3.4: Hardware Requirements for Lingua Sign .....                               | 20 |
| Table 3.5: Software Requirements for Lingua Sign.....                                | 21 |
| Table 6.3 : Lingua Sign Technology Stack .....                                       | 40 |
| Table 6.6 : Lingua Sign Rest Api Endpoints.....                                      | 42 |
| Table 6.8.2 Training hyper-parameters for alphabet and word recognition models ..... | 43 |
| Table 6.8.5 : Performance Metrics Summary .....                                      | 45 |

## LIST OF ABBREVIATIONS

|               |                                                                  |
|---------------|------------------------------------------------------------------|
| <b>AI</b>     | Artificial Intelligence                                          |
| <b>ANN</b>    | Artificial Neural Network                                        |
| <b>API</b>    | Application Programming Interface                                |
| <b>ASL</b>    | American Sign Language                                           |
| <b>ASGI</b>   | Asynchronous Server Gateway Interface                            |
| <b>CORS</b>   | Cross-Origin Resource Sharing                                    |
| <b>CPU</b>    | Central Processing Unit                                          |
| <b>CV</b>     | Computer Vision                                                  |
| <b>DL</b>     | Deep Learning                                                    |
| <b>DNN</b>    | Deep Neural Network                                              |
| <b>FPS</b>    | Frames Per Second                                                |
| <b>GPU</b>    | Graphics Processing Unit                                         |
| <b>GRU</b>    | Gated Recurrent Unit                                             |
| <b>H5</b>     | Hierarchical Data Format version 5 (Keras model format)          |
| <b>HTTP</b>   | Hypertext Transfer Protocol                                      |
| <b>IDE</b>    | Integrated Development Environment                               |
| <b>JSON</b>   | JavaScript Object Notation                                       |
| <b>ML</b>     | Machine Learning                                                 |
| <b>MLP</b>    | Multilayer Perceptron                                            |
| <b>MJPEG</b>  | Motion Joint Photographic Experts Group                          |
| <b>MP</b>     | MediaPipe                                                        |
| <b>PSL</b>    | Pakistani Sign Language                                          |
| <b>RAM</b>    | Random Access Memory                                             |
| <b>ReLU</b>   | Rectified Linear Unit                                            |
| <b>REST</b>   | Representational State Transfer                                  |
| <b>RNN</b>    | Recurrent Neural Network                                         |
| <b>SLR</b>    | Sign Language Recognition                                        |
| <b>SRS</b>    | Software Requirements Specification                              |
| <b>SQLite</b> | Structured Query Language Lite (lightweight relational database) |
| <b>TTS</b>    | Text-to-Speech                                                   |
| <b>UI</b>     | User Interface                                                   |
| <b>UX</b>     | User Experience                                                  |
| <b>WLASL</b>  | World Level American Sign Language                               |
| <b>YOLO</b>   | You Only Look Once                                               |



# **CHAPTER 1**

## **INTRODUCTION**

## 1.1 INTRODUCTION

One of the most basic human rights is communication. However, speech and hearing problems prevent millions of people from communicating. The United Nations estimates that 72 million people who are deaf or have speech impairments communicate largely through sign languages [1]. Globally, there are about 300 sign languages, each with its unique body language, hand gestures, facial emotions, vocabulary, and syntax [1]. Despite being full and expressive natural languages, a large portion of the hearing population is still unfamiliar with sign languages, which creates serious communication difficulties.

This issue is especially important in Pakistan. Due to a lack of qualified sign language interpreters, low public awareness of Pakistani Sign Language (PSL), and a lack of technology support, the hearing-impaired group faces social marginalization. These communication obstacles restrict deaf people's ability to fully participate in society by limiting their access to public services, healthcare, education, and work. Conventional communication techniques, such as taking notes and lipreading, are frequently ineffective, time-consuming, and unreliable.

Recent developments in computer vision, deep learning, and artificial intelligence (AI) have greatly enhanced automatic sign language recognition systems [2]–[6]. By converting sign language motions into text or speech, camera-based identification systems offer an affordable substitute for sensor-based methods. However, the majority of current systems are not well suited to regional sign languages like Pakistani Sign Language (PSL) because they were created for widely used sign languages like American Sign Language (ASL) [2]–[4]. Current PSL recognition systems frequently have drawbacks, such as limited real-time performance, reliance on specialized hardware, or support for just static gestures [7]–[9].

In this project, we introduce *Lingua Sign*, a full-stack AI-based online application that can recognize and translate Pakistani Sign Language in real time. The system records live video using a standard webcam, uses Media Pipe [11] to extract 21 hand landmark coordinates, classifies motions using a tiny Multilayer Perceptron (MLP) neural network created in TensorFlow [12], and generates text and synthetic Urdu voice. The proposed approach achieved 98.28% classification accuracy on a test dataset of 37 PSL alphabet classes while operating entirely offline, making it suitable for use in low-resource scenarios.

## 1.2 Background

Sign Language Recognition (SLR) is a fast-growing research field aiming to enable deaf and hard of hearing people to communicate effectively with the general population. Deep learning had a huge impact in the field. Early detection systems relied on hand-crafted feature extraction and traditional machine-learning classifiers which are not good at generalizing to different signers, lighting conditions and environments.

Deep learning techniques have been driving a lot of advances in today's Sign Language Recognition (SLR) systems, allowing for more precise, robust and real-time recognition of hand gestures. While traditional machine learning methods rely on hand-crafted features, deep learning models can automatically learn hierarchical representations from large-scale image and video data. This ability enables them to handle variations in hand shape, orientation, lighting conditions, background clutter and differences between signers. They are well suited for real world applications.

Convolutional Neural Networks (CNNs) have become one of the most popular architectures in the sign language recognition domain due to their excellent capability to extract spatial features from images and video frames. CNNs automatically learn salient visual features such as hand contours, finger poses, textures, and motion patterns without the need for hand-crafted feature extraction. This stacked architecture allows the model to learn more complex representations. This results in high recognition accuracy even under difficult conditions. For better recognition performance with lower computational complexity, popular CNNs such as Res Net , Efficient Net , Mobile Net and Vision Transformers ( Vits ) are used.

Although CNNs are good at capturing spatial information, many sign language gestures involve motion over time and therefore temporal modelling is equally important. Recurrent Neural Networks (RNNs) like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks are built to handle sequential data, keeping track of information from previous frames. These architectures can learn the temporal dependencies between successive hand motions, so that they can distinguish gestures that are visually similar in single frames but differ in their motion patterns. Bidirectional LSTMs even enhance the results by looking at the gesture sequences in both directions, thus yielding more context.

In this project we try to address these challenges by developing a complete system for Pakistani Sign Language recognition and translation by integrating the whole processing pipeline. The proposed system includes: robust gesture acquisition, efficient preprocessing, real-time hand

landmark extraction using Media Pipe, deep learning-based gesture classification, text generation, and speech synthesis in a responsive web application. The rich and varied PSL data set is used to improve the model's generalization ability and recognition accuracy for different users and environments. The system also takes into account usability, accessibility and scalability. It provides an intuitive interface to enable communication between sign language users and non-signers. The project combines state-of-the-art deep learning approaches with an efficient deployment framework to make Pakistani Sign Language technology more practical and accessible for real world applications.

### **1.3 Problem Statement**

People with hearing and speech impairments in Pakistan face a major communication barrier .At the moment, there is no reliable and affordable system widely available that can instantly translate gestures of Pakistani Sign Language into text or synthesized speech. The current solutions suffer from one or more of the following critical limitations:

- Limited gesture coverage: Many systems only recognise a limited range of static hand shapes (e.g., the alphabet) and do not support dynamic words or sentences.
- Hardware dependency: A few solutions require specialized glove-based sensors or Microsoft Kinect devices, which are costly and not suitable for daily use.
- Connectivity requirements: Certain cloud-based solutions demand uninterrupted internet connectivity, and thus may not be suitable for locations with sub-standard network infrastructure
- Limited robustness: Current systems are not robust to various lighting conditions, background and signers, restricting their real-world applicability
- No Urdu speech output: The accessibility for non-English users is low because there is no publicly available system that produces synthesized Urdu speech from PSL gestures.

Such restrictions limit the deaf community's access to education, healthcare and social interaction. Therefore, there is a dire need for the development of a real-time, robust, and offline camera-based PSL recognition system for gesture translation into Urdu speech and text.

### **1.4 Objectives**

The primary objectives of this project are as follows:

1. To develop an AI based system which can recognize Pakistani Sign Language gestures in real-time using a general-purpose webcam without any specialized hardware.

2. To translate recognised gestures into comprehensible text and synthesised Urdu speech output to enable efficient communication between deaf and non-signer people.
3. To create and build an extensive PSL dataset of 37 alphabet classes and a set of common word gestures captured under different lighting and environmental conditions.
4. To build the system as an offline-capable, full stack web application that can operate without an internet connection, ensuring that it can be used in diverse environments.
5. To add a sentence builder feature that builds grammatically meaningful utterances from individual recognised letters and words.
6. To design a scalable and modular architecture that allows for future expansion to other gestures, sign languages and deployment platforms.

## **1.5 Scope of the Project**

This project is on design, development and evaluation of real time sign language recognition system based on Pakistani Sign Language. The system recognizes a fixed set of static alphabetic gestures (37 classes) and a small set of dynamic word gestures. It is a full stack web app with a Python Fast Api backend and a Next.js 15 frontend talking over a local REST Application Programming Interface (API).

This system is designed for educational, social and assistive communication use. It does not replace professional sign language interpreters. It does not enable full continuous sign to sentence translation at the discourse level. The project does not include, among others, emotional recognition by facial expression analysis, body posture interpretation beyond hand landmark extraction, and complex contextual Natural Language Processing (NLP) for sentence-level semantics.

# **CHAPTER 2**

## **LITERATURE REVIEW**

## 2.1 LITERATURE REVIEW

In this chapter, we present the state-of-the-art research works of sign language recognition in which deep learning techniques, landmark-based approaches, and Pakistani Sign Language related studies are discussed. This review highlights the main trends, technical contributions and research gaps underlying this work.

## 2.2 Overview of Sign Language Recognition Systems

Based on the sensing technology employed, Sign Language Recognition (SLR) systems are often divided into three groups: glove-based, depth sensor-based, and vision-based. Wearable sensors, such as flex sensors and inertial measurement units (IMUs), are used by glove-based systems to record hand orientation and finger movements. These systems are costly, difficult to wear, and inappropriate for daily usage, despite their excellent identification accuracy [6]. To enhance gesture recognition, depth sensor-based systems—like those utilizing Microsoft Kinect—capture three-dimensional hand and body movements. However, their practical application is limited since they need controlled settings and specialized gear [7].

Because vision-based systems make advantage of the common RGB cameras found in laptops, webcams, and smartphones, they have become the most popular method. These systems are inexpensive, simple to set up, and don't require users to wear extra hardware. The accuracy and real-time performance of vision-based sign language recognition systems have been greatly enhanced by recent developments in deep learning and computer vision [2]–[5], [8].

In vision-based sign language recognition, there are two popular methods for feature representation. Convolutional Neural Networks (CNNs) are used directly to process raw or preprocessed images in the first image-based approach. Without the need for manual feature engineering, CNNs automatically pick up spatial features such as hand shape, finger placements, and gesture patterns [2], [3], [5].

The second is the landmark-based method, which makes use of hand keypoints that pose estimate frameworks like MediaPipe have retrieved. This method creates a compact and organized feature representation by representing motions using the coordinates of hand landmarks rather than processing the entire image. Landmark-based techniques are ideal for real-time applications because they are computationally efficient and less susceptible to changes in lighting and background clutter. As a result, they are now frequently used in

contemporary sign language recognition systems, such as the method used in this experiment [11].

### **2.3 International Sign Language Recognition Research**

International research in Sign Language Recognition (SLR) has advanced considerably over the past decade, particularly for American Sign Language (ASL) and other widely used sign languages. The fast growth of deep learning, computer vision and pose estimation techniques has allowed researchers to design a very accurate recognition systems that can work in real time. These studies have established benchmark datasets and modern architectures that continue to influence sign language research worldwide.

Nasharudin presented a deep learning model based on YOLO for the recognition of American Sign Language alphabets, reaching an accuracy of around 86%[2]. Their work proved that object detection models can be successfully applied to hand gesture recognition while retaining real-time performance. This technique reduced the processing time and showed the potentiality of YOLO architectures for practical applications in sign language.

Anand and Kumari have proposed a hybrid CNN-LSTM model with attention mechanism for isolated word recognition using the World Level American Sign Language (WLASL) dataset. They used CNN for spatial feature extraction and LSTMs for temporal modeling. Their model achieved an accuracy of 84.65%[5]. With the addition of the attention mechanism, which enables the network to attend only to the most informative frames in a gesture sequence, the performance was further improved.

The glove-based systems can achieve very high accuracy in measuring gestures, but require special hardware which makes them less feasible for everyday communication. The international studies presented in this paper together show the power of deep learning, temporal modeling and multimodal approaches and can be useful for the development of modern Pakistani Sign Language recognition systems.

### **2.4 Pakistan Sign Language Recognition Research**

There is very little research into SL in Pakistan but it is increasing in recent years. Halim and Abbas designed the Kinect based hand gesture recognition system for PSL. They proved that it is reasonably accurate to track PSL signs with limited vocabulary using depth perception. The system, however, is limited to the Kinect hardware platform and consequently its applicability is limited. Hamza and Wali [8] have built deep learning models to recognize sign

languages with a small PSL dataset, showing that transfer learning and compact architectures might be effective in addressing the limitation in training data. They pointed out that to them the greatest challenge was the need for well-annotated PSL corpora.

Imran et al. created a set of hand landmarks and applied machine learning classifiers to recognize the PSL hand configurations. The data and approach were useful as a jumping-off point in subsequent work, but they focused on prototyping and did not consider the deployment or speech output. To the best of our knowledge there is no complete and deployable system to be used in all PSL studies that is based on real time gesture detection, bilingual text and speech output, offline operation and user friendly interface. The present work is to remedy this.

## **2.5 Deep Learning Techniques in Sign Language Recognition**

In recent years transformer models which were developed in the field of natural language processing (NLP) have been transferred to the field of sign language recognition (SLR) and have demonstrated state-of-the-art performance on different benchmark datasets. Transformers are able to model long-range temporal dependencies because they do not suffer from the vanishing gradient problem, unlike traditional Recurrent Neural Networks (RNNs). They also incorporate a self-attention mechanism to allow them to pay attention to the most informative frames in a gesture sequence, enhancing the recognition of complex and dynamic gestures. One of the new developments implemented by SLR is its ability to detect landmarks in real-time by using only one RGB camera with the help of Google's MediaPipe Holistic framework. It records 21 hand landmarks, 33 body pose keypoints and 468 facial landmarks, providing a detailed description of the signer's expressions and movements. It is not based on images, but on the landmark coordinates, making MediaPipe more robust to lighting, skin color, resolution of background and images changes. The combination of Transformer models with MediaPipe has taken sign language recognition systems to a new level in today's world. The Usefulness of landmark based features to reduce the computational cost and usefulness of Transformers for temporal relation modelling between gestures. In addition, MediaPipe's lightweight design makes it possible to run on top of the standard computer and mobile devices without any special set of GPU hardware, making it ideal for resource and real-time constrained applications

## **2.6 Comparative Analysis and Research Gap**

Table 2.6 provides a comparative summary of related work in sign language recognition, highlighting the modality, technique, target language, and key limitations of each study.

| Reference            | Modality     | Technique            | Language    | Accuracy | Limitation                    |
|----------------------|--------------|----------------------|-------------|----------|-------------------------------|
| Nasharudin et al.[2] | RGB Camera   | YOLO-based DNN       | ASL         | 86%      | Static only; no speech output |
| Anand & Kumari[3]    | RGB Video    | CNN-LSTM + Attention | ASL (WLASL) | 84.65%   | High computation cost         |
| Halim & Abbas[7]     | Kinect Depth | ML Classifiers       | PSL         | N/A      | Specialized hardware required |
| Hamza & Wali[8]      | RGB Camera   | Deep Learning        | PSL         | N/A      | Small dataset; no real-time   |
| Imran et al.[9]      | RGB Camera   | ML + Landmarks       | PSL         | N/A      | Static only; no TTS output    |
| LinguaSign (Ours)    | RGB Camera   | MLP + MediaPipe      | PSL         | 98.28%   | Dynamic gestures              |

Table 2.6: Comparative Summary of Related Sign Language Recognition Systems

This analysis shows that no existing PSL system has all of the following: real time operation on standard hardware, offline capability, Urdu speech synthesis, a sentence builder and a user friendly full stack web interface. LinguaSign is designed to meet all these requirements at once.

# **CHAPTER 3**

## **SYSTEM REQUIREMENTS SPECIFICATION**

## **3.1 SYSTEM REQUIREMENTS SPECIFICATION**

This chapter specifies in detail the requirements of the LinguaSign system. It describes the overall system, functional requirements, non-functional requirements, and hardware and software specifications required to implement and run the system.

### **3.2 Overall System Description**

Lingua Sign is an AI based Pakistani Sign Language recognition and translation platform that aims to enable real time communication between the deaf or speech impaired and non signers. The system uses a regular webcam to recognize hand gestures. Media Pipe extracts 21 hand landmark coordinates from each frame, and an MLP neural network trained on this data classifies the gesture. The predicted gesture is visualized on screen as text and also translated to Urdu speech using Microsoft Edge Neural TTS.

The system can work in two recognition modes. In the Alphabet Mode, gestures are classified into 37 PSL alphabet classes . In Word Mode, a set of full-word gestures are recognized. Lingua Sign is a full stack web application. It has a Python Fast API backend and a Next.js 15 typescript frontend. All processing occurs on the user's local machine, allowing for offline and private use.

### **3.3 Functional Requirements**

Functional requirements are the specific functions and operations that the LinguaSign system must perform.

#### **3.3.1 User Requirements**

- The system shall be capable of capturing hand gestures from a live camera feed without and additional hardware
- The system shall correctly identify static gestures of PSL alphabet (37 classes) with accuracy >90%
- The system shall recognize dynamic PSL word gestures by a model trained separately.
- The system shall recognize the gesture to convert it to readable text and display it on the screen in real time.
- The system shall convert the types text into Urdu speech using a neural Text-to-Speech engine.
- The system shall include a sentence builder that will build complete utterances from the individual letters and words identified.

### **3.3.2 System Requirements**

- The system shall process live video frames in real time at a minimum rate of 15 fps.
- Media Pipe Holistic will be used to extract 21 hand landmark coordinates (x, y per landmark) from each frame.
- The system shall classify the extracted feature vector using a pre-trained Keras MLP model loaded at system startup.
- The system shall provide a REST API for the frontend to start/stop capture, request predictions and trigger speech synthesis.
- The system shall stream a live annotated MJPEG video feed to the frontend with Media Pipe landmarks overlaid.

### **3.3.3 Data Processing Requirements**

- The system shall normalize landmark coordinates for scale and translation invariance.
- The system shall apply a pre-fitted Standard Scaler to feature vectors prior to inference, to maintain consistency with training.
- SQLite database will be used by the system to save the training data with separate tables for alphabet and word datasets.
- The system shall process gesture data without storing user video or landmark data persistently.
- The system shall provide dataset augmentation via specific capture scripts.

### **3.3.4 Gesture Recognition Requirements**

- The system shall recognize all 37 PSL alphabet gestures including unique gestures of Urdu script .
- The system shall be able to add new PSL gestures in the future without architectural changes.
- The system shall output a confidence score with each prediction to gracefully handle unclear gestures.
- The system shall change between Alphabet Mode and Word Mode at runtime without restarting the capture pipeline.

### **3.3.5 Dashboard and Progress Tracking**

- The total number of recognized signs during practice sessions shall be displayed on the dashboard.
- The system shall keep track of a daily practice streak to encourage consistent learning.
- The dashboard will log and display the user's recent learning history and signs identified.
- The system shall show weekly activity summary to help the user to evaluate his/ her learning consistency.
- Dashboard should show visual statistics and progress to encourage continued practice.
- User progress data will be stored locally (in the browser's Local Storage) for privacy and offline access.
- The dashboard shall update the learning statistics automatically after each practice session.
- The system shall remember previously stored progress information across multiple application sessions.
- The dashboard will allow users to monitor their learning progress and areas for further improvement.

### **3.3.6 Settings and User Preferences**

- The system will allow the user to set the application preferences in a Settings page.
- Users can switch between Light Mode and Dark Mode for enhanced accessibility.
- Users should be able to define their interface preferences according to their needs.
- User preferences are stored locally in the browser, so they are available across sessions.
- The system should automatically fetch the saved user preferences when the application is launched.
- The Settings page will be a simple and intuitive interface that allows user to change application preferences
- Users settings are private and we never send them to external servers.

## **3.4 Non-Functional Requirements**

### **3.4.1 Performance**

- Gesture processing and classification must be completed within 1 second per detection cycle under normal operating conditions.

- MJPEG video stream must be provided at minimum of 15 FPS for smooth user experience.
- The system must support continuous video input for long periods of time without memory leaks or performance loss.

#### **3.4.2 Accuracy**

- The alphabet recognition model shall achieve a minimum of 90% test accuracy on the held out evaluation set.
- Use data augmentation and balanced class distribution in the training dataset to reduce misclassification rates.
- The model should generalize to multiple unseen users during testing.

#### **3.4.3 Reliability**

- The system will perform consistently in a variety of lighting conditions (ambient indoor, direct sunlight, low light).
- The system shall be robust against transient camera errors and MediaPipe detection failures.
- Offline caching of TTS audio will enable sub-second speech playback of already synthesized text.

#### **3.4.4 Usability**

- The interface should be usable by users with no previous technical training within five minutes.
- Clearly labelled on-screen buttons for start/stop detection, accept letter and speak sentence for easy access.
- The system shall provide light and dark themes for varying visual preferences and environments.

#### **3.4.5 Scalability**

- Environment variables for backend configuration for various camera indices, model versions and deployment paths
- The architecture of the system should allow for the integration of new sign languages without any changes in the structural code, through retraining of the model.

- The re-training pipelines (retrain.py) should be able to handle incremental dataset growth.

### 3.4.6 Security and Privacy

- No user data will be sent to any external servers, all video and gesture data will be processed locally.
- Backend will implement Cross-Origin Resource Sharing (CORS) controls to allow access only from trusted origins.
- Trained model artifacts should be saved in a secure, non-public directory and loaded in read-only mode at runtime.

## 3.5 Hardware Requirements

The following hardware specifications are required for the development and operation of Lingua Sign.

| Component | Minimum Specification    | Recommended Specification                 |
|-----------|--------------------------|-------------------------------------------|
| Camera    | HD Webcam (720p)         | Full HD Webcam (1080p, 30 FPS)            |
| Processor | Quad-Core CPU @ 2.0 GHz  | AMD Ryzen 5\7                             |
| RAM       | 8 GB                     | 16 GB                                     |
| GPU       | Optional (CPU inference) | 1660 Super 8GB                            |
| Storage   | 10 GB free space         | 50 GB free space (for dataset and models) |
| Audio     | Integrated speakers      | External speakers or headphones           |

Table 3.4: Hardware Requirements for Lingua Sign

## 3.6 Software Requirements

| Component        | Specification  |
|------------------|----------------|
| Operating System | Windows 10/11. |

|                                |                                                          |
|--------------------------------|----------------------------------------------------------|
| Programming Language (Backend) | Python 3.9 or 3.10.                                      |
| Web Framework (Backend)        | FastAPI with Uvicorn ASGI server                         |
| Machine Learning Libraries     | TensorFlow 2.x / Keras, scikit-learn                     |
| Computer Vision                | OpenCV 4.x, MediaPipe 0.10+                              |
| Text-to-Speech                 | edge-tts (Microsoft Edge Neural TTS), pyttsx3 (fallback) |
| Frontend Framework             | Next.js 15, TypeScript, React 18, Tailwind CSS           |
| Database                       | SQLite 3 (via Python sqlite3 module)                     |
| IDE                            | VS Code, PyCharm,                                        |
| Browser                        | Google Chrome , Mozilla Firefox , or Microsoft Edge.     |

Table 3.5: Software Requirements for Lingua Sign

# **CHAPTER 4**

## **SYSTEM DESIGN AND MODELING**

## 4.1 SYSTEM DESIGN AND MODELING

In this chapter the entire design of the LinguaSign System is described. It consists of overall architecture design, then detailed design models such as Use Case Diagram, Activity Diagram, Class Diagram, Data Flow Diagrams and System Sequence Diagram. These models together provide different views of the structure, behavior and data interactions in the system.

### 4.2 Architectural Design

The LinguaSign system is built on a modular, layered client-server architecture, which guarantees system scalability, maintainability and real-time performance. The architecture is divided into five functional layers each one of which is responsible for a certain aspect of how the system will function:

- The Input Layer – Reads the current video frames from the user's webcam with OpenCV and sends them to the processing pipeline.
- Landmark Extraction Layer: Runs MediaPipe Holistic on each frame to extract 21 landmarks for the right hand (x, y per landmark) and 33 keypoints for body pose. Landmarks extracted are then serialized into json files that are fed to downstream.
- Machine Learning Layer: Reads the landmark JSON (which is already serialized), applies the pre-fitted StandardScaler, and does a forward pass through the loaded Keras MLP model. The predicted class label is then used to decode it using LabelEncoder and recognized as gesture.
- Application Layer: The FastAPI backend handles the management of the capture lifecycle, inference, speech synthesis and MJPEG streaming. The Next.js front end polls the back end, controls the state of the sentence builder, and displays the annotated video feed for the user.
- Output Layer: Generates gesture prediction as text on screen, builds words and sentences with the help of sentence builder and generates Urdu speech by using Microsoft Edge Neural TTS

There are a number of design principles that have guided the architecture, such as modularity (each component can be deployed independently of one another), privacy (all processing will take place locally, without any data being transmitted to the outside), scalability (new gestures and languages will be added to the system without the need to redesign the pipeline), and cross-platform compatibility (the web application will run on any device that has a modern browser and webcam).

Figure 4.2 illustrates the overall Lingua Sign system architecture.

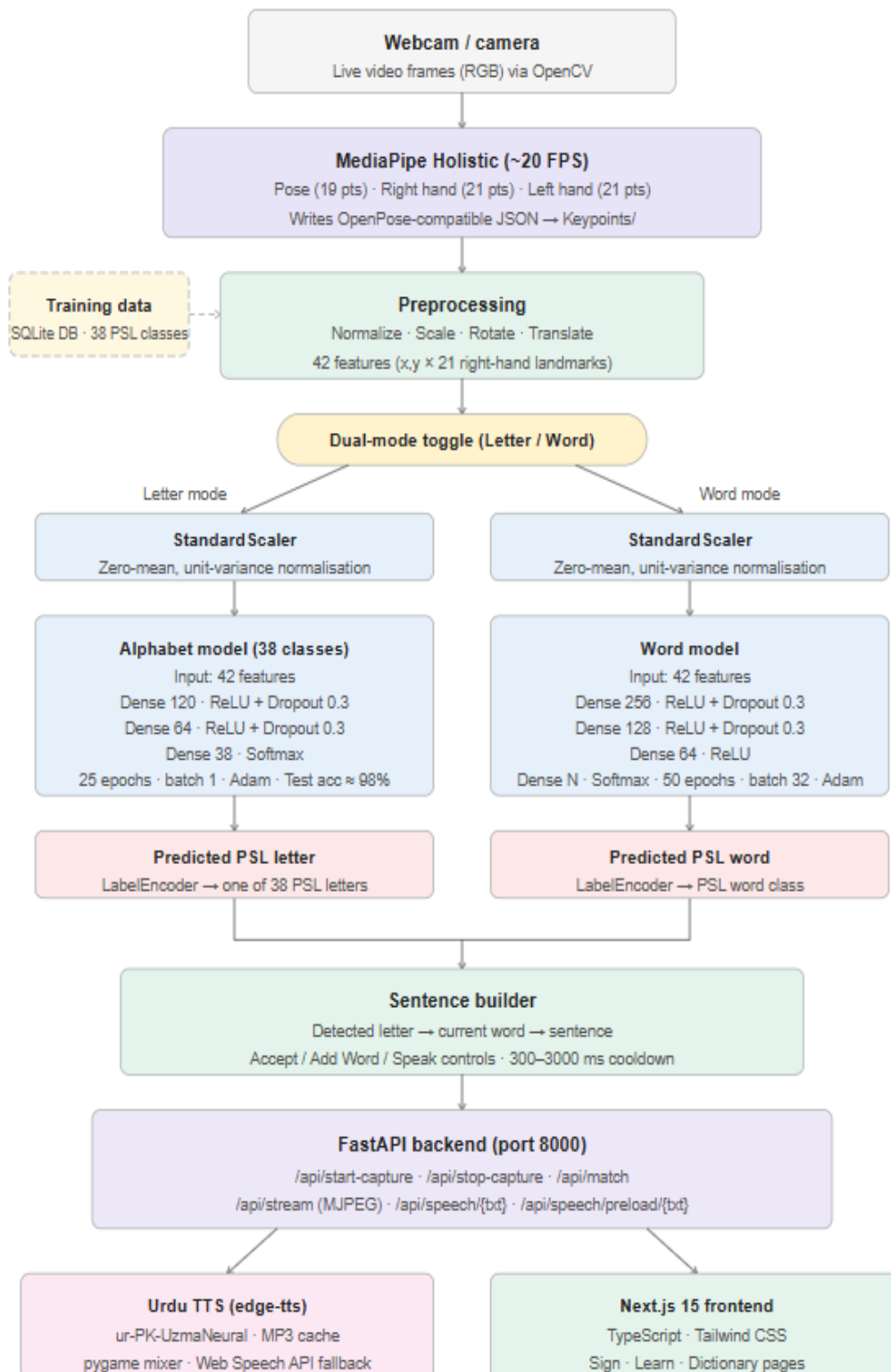


Figure 4.2: Lingua Sign System Architecture Diagram

## 4.3 Design Models

In this section we present the UML and data flow models used to specify the behavioural and structural properties of the Lingua Sign system. Each model is characterized by purpose and key elements.

### 4.3.1 Use Case Diagram

The Use Case Diagram shows the external actors of the system and the functional use cases they are involved in. We identify three main actors: the Deaf User, who uses the system for real-time sign-to-speech translation and learning, the Normal User, who uses the system to understand PSL gestures, and the Administrator, who manages the dataset and retrain recognition models. Main use cases are Start Sign Detection, Accept Letter, Build Sentence, Speak Sentence, Learn Signs, Browse Dictionary, Capture Training Data, Retrain Model. The Use case diagram shown in Figure 4.3.1

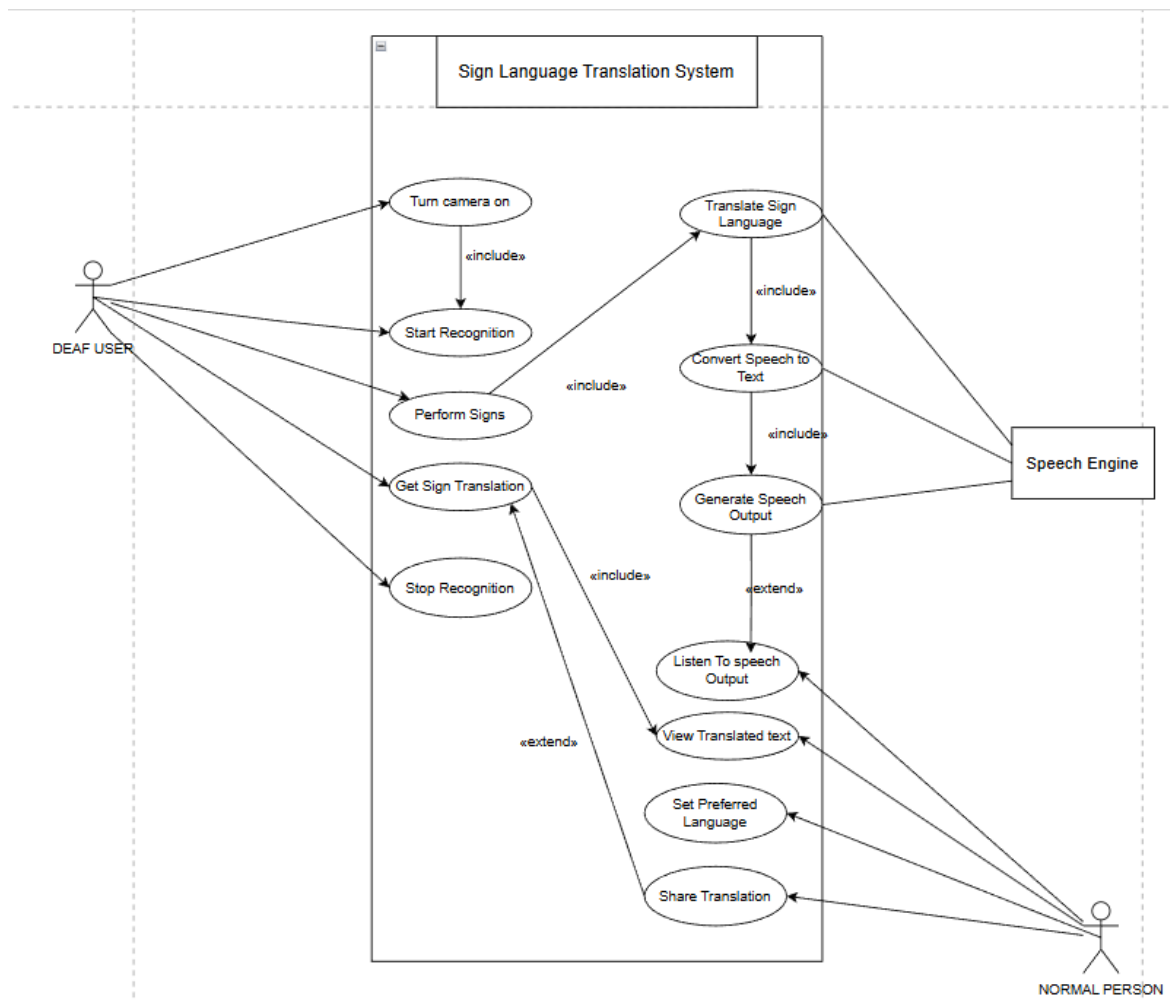


Figure 4.3.1: Use Case Diagram of Lingua Sign System

### 4.3.2 Activity Diagram

As seen in Figure 4.3.2, the activity diagram depicts the general process of the suggested Pakistani Sign Language (PSL) recognition system. When the user opens the application and chooses between Real-time and Offline mode, the procedure starts. While the system processes a pre-recorded gesture video in offline mode, it uses a webcam to record live gestures in real-time mode.

Before Media Pipe extracts the hand landmarks, the collected frames are pre-processed. The machine learning model for gesture categorization is then given the retrieved landmark coordinates and generated features. The anticipated sign is translated into text and shown on the screen if the gesture is correctly identified. The Text-to-Speech (TTS) module is then used to turn the generated text into speech, and the user hears the appropriate audio. The user can repeat the identification process if the gesture is not recognized by the system, which then shows an error or retry message.

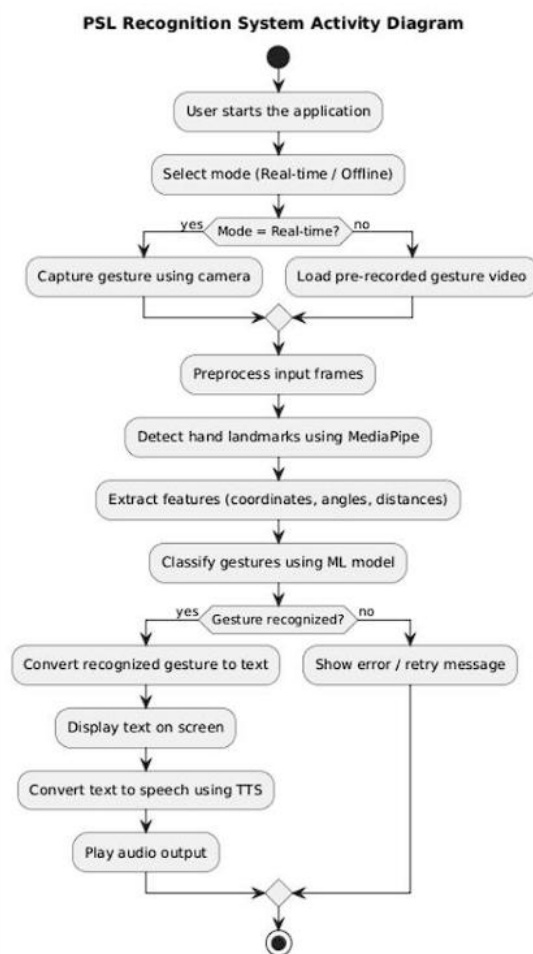


Figure 4.3.2: Activity diagram of Lingua Sign System

### 4.3.3 Class Diagram

The Class Diagram describes the structure of the backend of LinguaSign, showing the main classes, their attributes and methods, and the relationships between classes. Main classes are: Capture Service, responsible for OpenCV camera loop and Media Pipe Holistic processing; Recognition Service, responsible for feature extraction, scaling and MLP inference; Speech Service, responsible for TTS synthesis and audio caching; fast API Router classes (Capture Router, Recognition Router, Speech Router, Stream Router), exposing the REST API endpoints. The main React components are modeled on the front-end side: Detection Page, Sentence Builder, and Learn Signs Page.

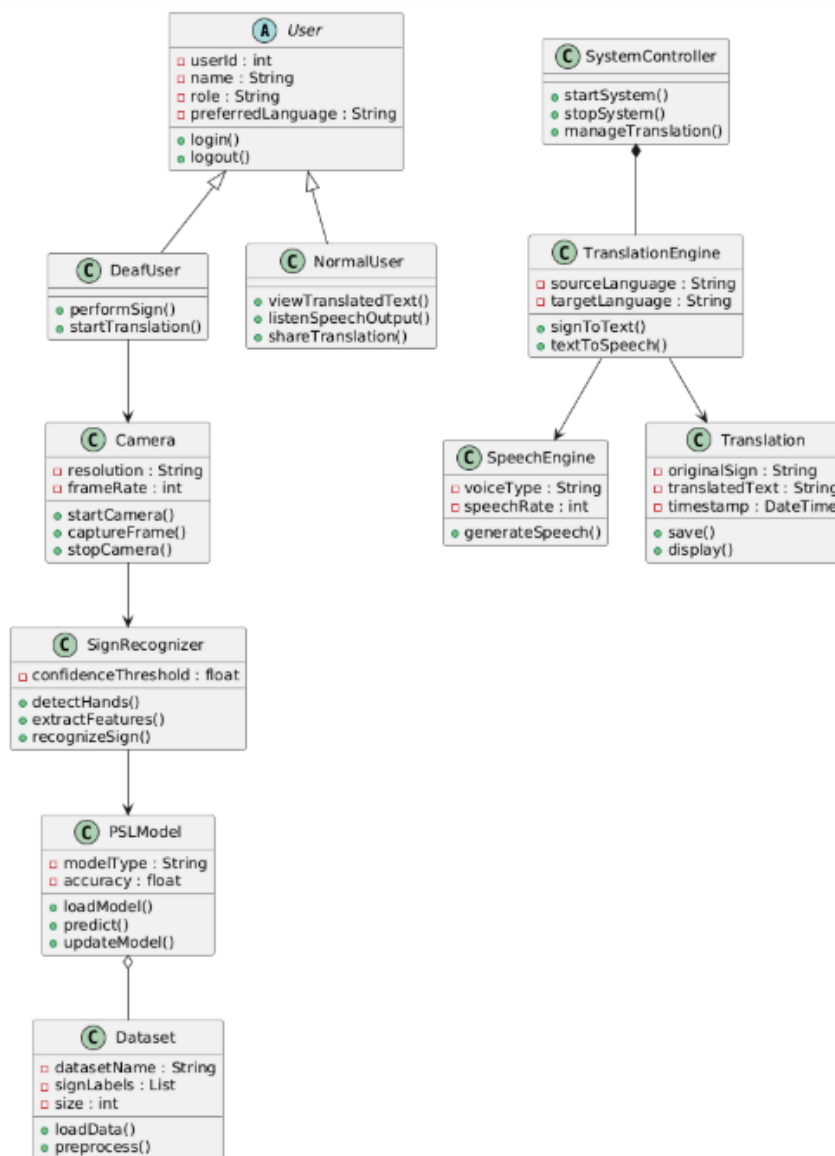
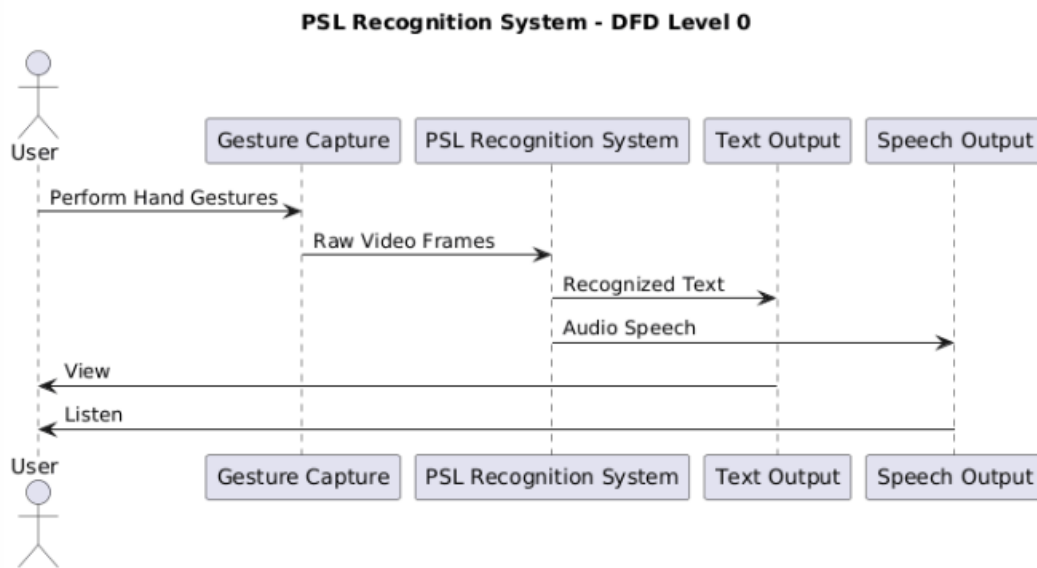


Figure 4.3.3: Class Diagram of Lingua Sign System

#### 4.3.4 Data Flow Diagram-Level 0

The Level 0 Data Flow Diagram (Context Diagram) depicts LinguaSign as a single process and shows the data flows between the system and the external entities. The inputs from the user are taken as Live Camera Feed and from the administrator are taken as Configuration Settings. The system generates two outputs: Recognized Text and Gesture Label, which are presented to the user, and Synthesized Urdu Speech, which is played through the audio output device.



. Figure 4.3.4 shows the Level 0 Data Flow Diagram.

#### 4.3.5 Data Flow Diagram-Level 1

The Level 1 Data Flow Diagram breaks the main system process into five sub-processes: (1) Video Capture and Frame Extraction, (2) MediaPipe Landmark Detection, (3) Feature Preprocessing and Normalization, (4) MLP Gesture Classification, and (5) Output Generation (Text and TTS). Data stores include: SQLite Training Database, Trained Model Store (H5, PKL files), and TTS Audio Cache. The diagram illustrates the flow of the landmark JSON files from capture process to classification process; and the flow of the predicted label to output generation process.

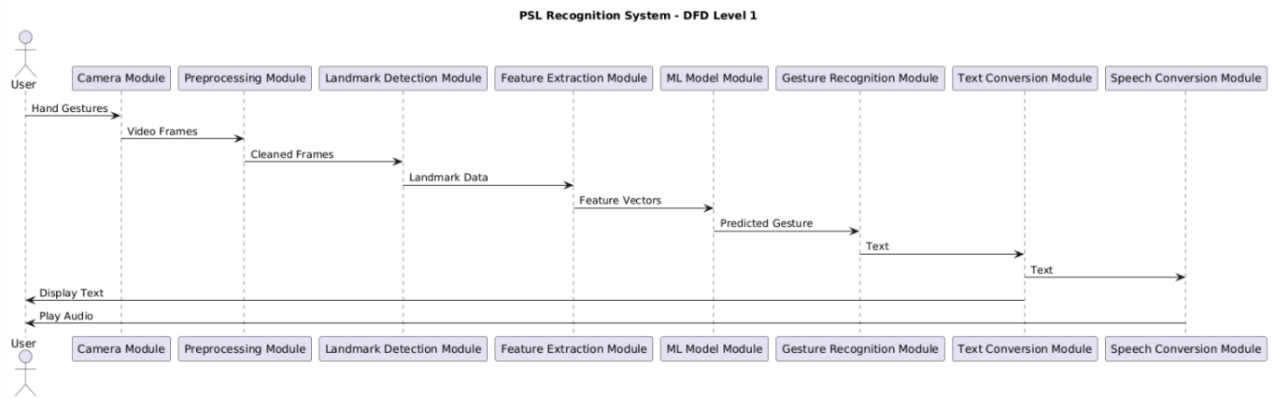


Figure 4.3.5 shows the Data Flow Diagram of Level 1.

### 4.3.6 System Sequence Diagram

The System Sequence Diagram (SSD) shows the interaction of the User with the Next.js Frontend, FastAPI Backend and MediaPipe Capture Thread during the detection process. The user hits Start Detection. The webcam goes on and starts processing. Frontend sends the keypoint data to backend to get prediction result and backend returns the detected label. If the letter is accepted, the sentence is changed and Speak Sentence provides Urdu speech output.

#### System Sequence

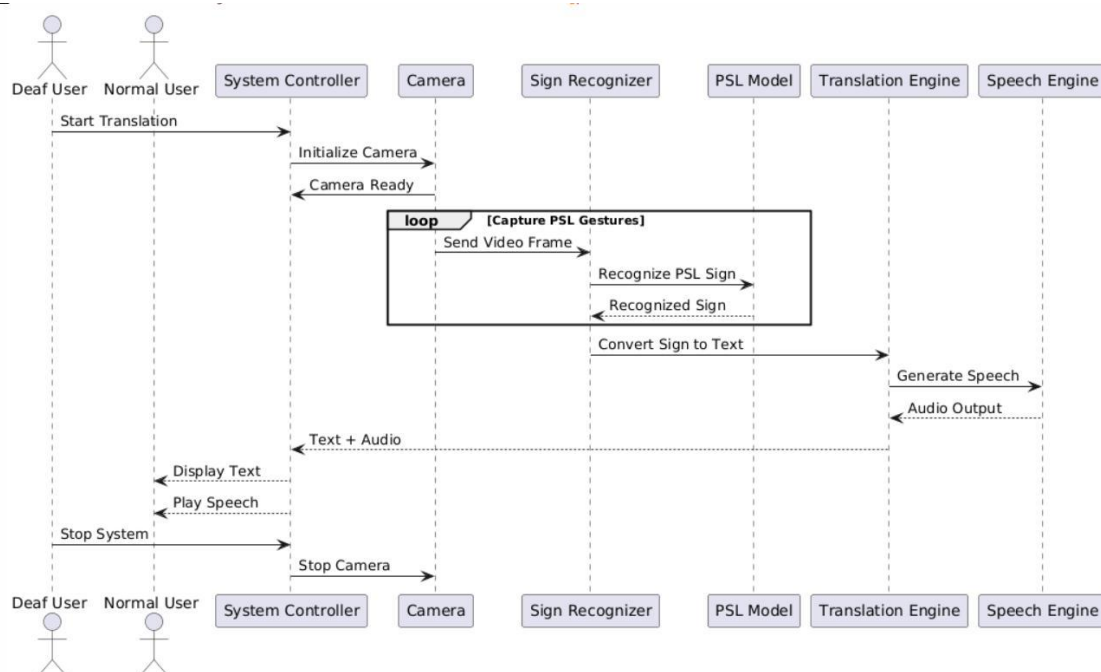


Figure 4.3.6: System Sequence Diagram

# **CHAPTER 5**

## **PROPOSED METHODOLOGY**

## 5.1 PROPOSED METHODOLOGY

In this chapter the step-by-step methodology behind the Lingua Sign system is presented, from the initial capture of a gesture to the final output of speech. The methodology is designed to address the key challenges of real-time performance, inter-signer variability, environmental robustness and offline operation. We describe in detail each pipeline stage, the techniques we used, and the design decisions that make Lingua Sign different from existing PSL systems.

### 5.2 Overall Methodology Overview

The Lingua Sign methodology is based on a six-step pipeline:

1. Data collection and dataset creation Systematic data collection of PSL gestures in various conditions and from multiple signers
2. Data Preprocessing: The obtained dataset is cleaned, normalized, augmented and stratified into train and test sets
3. Feature Extraction: A 42-dimensional compact landmark feature vector is computed from the Media Pipe hand detections.
4. Model development and training: Design, train and optimize MLP neural networks for alphabet and word recognition.
5. Sign-to-Text Conversion: Real time model prediction to text gesture label.
6. Text to Speech Conversion & System Integration: Synthesis of Urdu speech for combined sentences & end to end integration of all modules

### 5.3 Data Collection and Dataset Construction

Any machine-learning system needs a representative and balanced training data set as a basic precondition. To the best of our knowledge, there is no large scale public PSL dataset comparable to WLASL. Therefore, Lingua Sign constructs the training corpus with a custom data collection pipeline.

The gesture data is recorded by two scripts, `capture_alphabets.py` for 37 classes of PSL alphabets and `capture_words.py` for common word gestures. Each script opens the webcam, runs Media Pipe Holistic to get the hand landmarks and stores the landmarks coordinates as rows in the SQLite database (`data/db/main_dataset.db`). The database consists of two tables, alphabet Dataset and word Dataset. Each row contains the 21 right-hand landmark coordinate pairs ( $x_1, y_1$  through  $x_{21}, y_{21}$ ) and the class label it belongs to.

Dataset collection is performed under these protocols to maximize dataset diversity and model generalizability:

- Multi-signer contribution: To reduce signer-dependent bias, Dataset is contributed by at least three signers with different hand sizes and signing styles
- Lighting variation: Samples are chosen under indoor ambient light, direct sunlight and low light conditions to ensure robustness.
- Varied backgrounds. Sessions are conducted in front of plain, textured and cluttered backgrounds, avoiding the model learning background correlations.
- Variation in hand orientation: To increase robustness to orientation, each gesture is recorded with slightly different wrist angles and positions with respect to the camera.

The final dataset of each class has enough samples to train with an 80/20 train-test split and still be statistically valid. Data synthesis utility `PSL/helper/synthesize_data.py` provide additional geometric augmentation to further expand the effective training set.

| id     | x1 | y1  | x2  | y2                 | x3                 | y3                 | x4                 | y4                |
|--------|----|-----|-----|--------------------|--------------------|--------------------|--------------------|-------------------|
| 1      | 1  | 150 | 150 | 158.7802432212353  | 99.27647873028346  | 177.11691301224448 | 55.02986530111134  | 179.5888505563776 |
| 2      | 2  | 150 | 150 | 159.24561261423514 | 100.47195161014281 | 177.58023835450447 | 54.26243539061235  | 178.3753802029808 |
| 3      | 3  | 150 | 150 | 160.4065759798269  | 104.0301558906198  | 191.49607410167755 | 66.03298468870454  | 212.28575702131   |
| 4      | 4  | 150 | 150 | 160.26478772879216 | 102.98541739987894 | 190.53076983366083 | 66.65175730974862  | 211.629164036368  |
| 5      | 5  | 150 | 150 | 161.7894757291478  | 104.73967416842308 | 194.36469060384167 | 68.67781422775396  | 214.4858744925631 |
| 6      | 6  | 150 | 150 | 160.27733414189703 | 104.07530085170222 | 190.87982492121515 | 66.63114761893054  | 212.3126624759245 |
| 7      | 7  | 150 | 150 | 160.03006180018258 | 102.67849139183147 | 191.24386942356915 | 65.96047793561843  | 213.00956172214   |
| 8      | 8  | 150 | 150 | 157.99353695627383 | 102.45667592604173 | 188.20594524245544 | 63.56888653219862  | 210.216626922166  |
| 9      | 9  | 150 | 150 | 158.33630327447852 | 103.07574919371334 | 189.30652091004887 | 65.60955555017807  | 212.134121357799  |
| 10     | 10 | 150 | 150 | 158.29618822846496 | 103.78098033200814 | 187.61374346741874 | 65.89908419594785  | 210.894372889803  |
| 11     | 11 | 150 | 150 | 158.29798929163184 | 103.14259123229041 | 188.62214560252215 | 63.583685737577284 | 211.257331407067  |
| 12     | 12 | 150 | 150 | 159.7865237069139  | 103.5059496861951  | 191.17279416414107 | 64.09559932976127  | 211.901412518985  |
| 13     | 13 | 150 | 150 | 161.91772603116706 | 100.38422279271254 | 180.61178082564885 | 58.20981564512776  | 180.949603887620  |
| 14     | 14 | 150 | 150 | 159.62729889939206 | 103.9925544710039  | 190.9311469022495  | 65.17784643541538  | 212.46657121164   |
| 15     | 15 | 150 | 150 | 156.17992723690122 | 80.0438223028739   | 198.84337941674244 | 18.655634065132617 | 227.944000208930  |
| 16     | 16 | 150 | 150 | 160.80378220552863 | 104.27828696536207 | 191.46311820627577 | 65.43050534477021  | 212.678486063336  |
| 17     | 17 | 150 | 150 | 162.03968866776873 | 104.91768047523502 | 192.56374357475374 | 68.18341945122056  | 213.342845548294  |
| 18     | 18 | 150 | 150 | 162.35504415847108 | 103.99336182364635 | 193.94303076352742 | 66.7950581081102   | 213.725325406917  |
| 19     | 19 | 150 | 150 | 158.94148043478324 | 102.17158608862894 | 189.7855586591713  | 64.40982103889817  | 211.530071225077  |
| 20     | 20 | 150 | 150 | 158.881751065596   | 101.87348151131692 | 189.49665149367127 | 64.70478568705117  | 211.674938156797  |
| 21     | 21 | 150 | 150 | 159.9220318396424  | 103.76310969332889 | 190.1307224952203  | 66.81643643977294  | 212.024830322721  |
| 22     | 22 | 150 | 150 | 160.09114083052714 | 103.94686873053402 | 190.66117410001857 | 66.3686184268279   | 211.93067127420   |
| 15,364 | 23 | 150 | 150 | 159.19836680733006 | 102.76643603589335 | 189.56359279311565 | 65.99152617341518  | 211.7366941600008 |

Figure 5.3: Dataset Collection and construction

## 5.4 Data Preprocessing

The raw landmark data, collected from different signers and in different conditions, contains several sources of variability that need to be addressed before training the model. The following steps were applied.

- **Data cleaning:** Remove frames without detected hand using MediaPipe. We use interquartile range thresholding to eliminate sequences of landmarks that contain extreme outliers due to occlusion or detection jitter.
  - **Coordinate normalization:** Raw MediaPipe landmark coordinates are in the image pixel space, which depends on the resolution of the camera and the distance of the hand. We remove scale and translation effects by subtracting the wrist landmark (landmark 0) as the origin and by dividing by the maximum absolute coordinate value in each sample. This normalizes all landmarks to a unit bounding box centered at the wrist location.
  - **Standard scaler normalization:** A scikit-learn StandardScaler is fitted on the training set after coordinate normalization and applied to both the training and test sets. The scaler transforms each feature to have zero mean and unit variance. This helps to improve gradient flow during Training and to ensure consistent inference.
  - **Data augmentation:** To increase the effective size of the dataset and improve robustness, the helper utilities in PSL/helper/ apply (i) rotation augmentation, by randomly rotating the landmark cloud by small angles ( $\pm 15^\circ$ ) to simulate wrist rotation; (ii) scaling augmentation, by randomly scaling the landmark cloud by factors in the range [0.85, 1.15] to simulate varying hand distances; and (iii) horizontal mirroring, by reflecting the landmark cloud to simulate left hand equivalents.
  - **Label Encoding:** The class labels are encoded using the LabelEncoder from scikit-learn to map the string gesture names to integer indices. Then labels are one-hot encoded with `keras to_categorical` to be trained with categorical cross-entropy loss.
- Dataset split:** The data is split into train (80%) and test (20%) using stratified sampling (word dataset) to keep the class balanced in both splits. We set a random seed (42) for reproducibility.

All the preprocessing artefacts (the fitted StandardScaler and LabelEncoder) are serialized as pickle files (.pkl) and loaded with the model at a backend startup to ensure that the inference applies the same transforms to live data.

## 5.5 Feature Extraction

In fig 5.5 MediaPipe Holistic takes a frame as an input and returns three sets of landmarks (21 hand landmarks, for each hand) for that frame. Since the dominant hand is the one that mainly composes the gestures of PSL alphabet, LinguaSign only attends the 21 right-hand landmarks.

We represent each hand landmark as a normalized image-space 2D coordinate (x,y). For the classification we use only the x and y coordinates, thus each sample has a 42-dimension feature vector (21 landmarks \* 2 coordinates). Thus the 42 dimensional feature vector is sufficient to represent the complete two dimensional spatial configuration of the hand for classification between PSL alphabet classes.

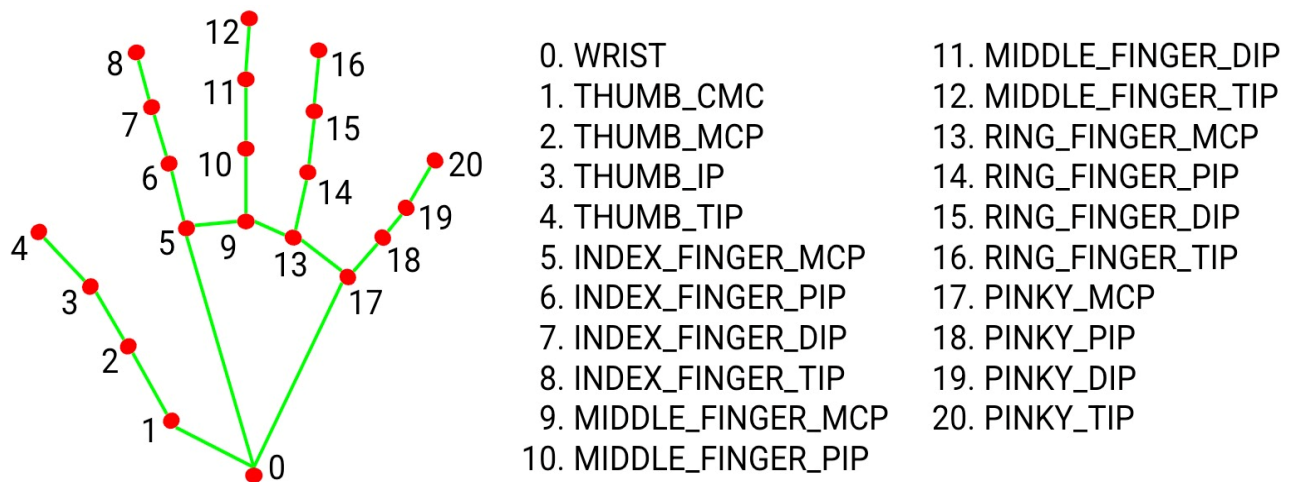


Figure 5.5: Feature Extraction

The hand landmark model in Media Pipe detects 21 points on a human hand. Every landmark is a tip or joint . Because the different finger joints used for tracking hand movement are CMC, MCP, PIP, DIP and TIP. The CMC is the thumb base joint attached to the wrist. The MCP are the knuckles. PIP is the joint in the middle of the finger. DIP is the joint near the tip of the finger. TIP is the tip of finger. The computer vision detects these key points from the webcam frames in real time and it gives the x, y coordinates of the hand. In Sign language detection system, the system detects finger movements and hand gestures by detecting the position, angle and distance between these landmarks. The extracted landmark data is then used by machine learning models to identify specific signs and convert them into text, speech or commands in real-time.

## 5.6 Model Development and Training

In Lingua Sign, there are two different MLP neural networks: one for Alphabet recognition and the other for Words recognition. Both models are coded in TensorFlow/Keras and trained on landmark feature vectors extracted from the pre-processed data set.

### 5.6.1 Alphabet Recognition Model

The Alphabet Recognition Model is a small feed-forward network used for the 37-class PSL alphabet classification task. The structure is as follows:

- Input Layer: 42 features (21 hand landmarks  $\times$  x, y)
- Hidden Layer 1: Dense (120 neurons, ReLU activation), Dropout(0.3).
- Hidden Layer 2: Dense(64 neurons, ReLU activation), Dropout(0.3).
- Output Layer: Dense (37 neurons, Softmax activation), one for each class in the PSL alphabet.

Dropout regularization (rate = 0.3) is used after each hidden layer to prevent overfitting on the training dataset and improve the generalization to unseen signers. The model is trained with Adam optimizer and categorical cross entropy loss. The training is done for 25 epochs with batch size 1. A custom EpochBar Keras callback provides real time progress tracking during training.

#### Input Layer

$x \in \mathbb{R}^{42}$  (21 landmarks \* x, y coordinates)

#### Hidden Layer 1 Dense(120, Relu) + Dropout(0.3)

$$z_1 = W_1 x + b_1 \quad W_1 \in \mathbb{R}^{120 \times 42}, b_1 \in \mathbb{R}^{120} \quad h_1 = \text{Relu}(z_1) = \max(0, z_1)$$

$$h_1^{\text{drop}} = \text{Dropout}(h_1, p = 0.3)$$

#### Hidden Layer 2 Dense (64, Relu) + Dropout(0.3)

$$z_2 = W_2 h_1^{\text{drop}} + b_2 \quad W_2 \in \mathbb{R}^{64 \times 120}, b_2 \in \mathbb{R}^{64}$$

$$h_2 = \text{Relu}(z_2) \quad h_2^{\text{drop}} = \text{Dropout}(h_2, p = 0.3)$$

### **Output Layer Dense(37, SoftMax)**

$$z_3 = W_3 h_2^{\text{drop}} + b_3 \quad W_3 \in \mathbb{R}^{37 \times 64}, b_3 \in \mathbb{R}^{37}$$

$$\hat{y}_i = \text{SoftMax}(z_3)_i = \exp(z_{3i}) / \sum_{j=1}^{37} \exp(z_{3j})$$

### **Dropout Regularization**

Dropout ( $h, p$ ) =  $h \odot \text{mask} / (1 - p)$ ,  $\text{mask} \sim \text{Bernoulli}(1 - p)$ , only applied during training.

### **Loss Function – Categorical Cross-Entropy**

$$L = - \sum_{i=1}^{37} y_i \cdot \log(\hat{y}_i)$$

where  $y_i$  is the one-hot ground-truth label and  $\hat{y}_i$  is the predicted probability for class  $i$ .

### **5.6.2 Word Recognition Model**

The Word Recognition Model employs a deeper architecture to capture the higher-complexity feature space of full-word gestures:

- Input Layer: 42 features (21 right hand landmarks \* x, y).
- Hidden Layer 1: Dense (256 neurons, activation=relu) + Dropout (0.3)
- Hidden Layer 2: Dense (128 neurons, Relu activation) + Dropout (0.3)
- Hidden Layer 3: Dense (64 neurons, Relu activation).
- Output Layer: Dense (Num classes, activation = SoftMax ) .

The word model is trained for 50 epochs with batch size of 32 and 80/20 stratified train-test split. The additional depth and width over the alphabet model indicate larger intra-class variation and larger semantic space of word-level gestures.

### **Input Layer**

$$x \in \mathbb{R}^{42} \text{ (21 landmarks} \times \text{x,y coordinates)}$$

### **Hidden Layer 1 -- Dense (256, Relu) + Dropout (0.3)**

$$z_1 = W_1 x + b_1 \quad W_1 \in \mathbb{R}^{256 \times 42}, b_1 \in \mathbb{R}^{256}$$

$$h_1 = \text{Relu}(z_1)$$

$$h_1^{\text{drop}} = \text{Dropout}(h_1, p = 0.3)$$

### **Hidden Layer 2 — Dense(128, Relu) + Dropout (0.3)**

$$z_2 = W_2 h_1^{\text{drop}} + b_2 \quad W_2 \in \mathbb{R}^{128 \times 256}, b_2 \in \mathbb{R}^{128}$$

$$h_2 = \text{Relu}(z_2)$$

$$h_2^{\text{drop}} = \text{Dropout}(h_2, p = 0.3)$$

### **Hidden Layer 3 — Dense (64, Relu)**

$$z_3 = W_3 h_2^{\text{drop}} + b_3 \quad W_3 \in \mathbb{R}^{64 \times 128}, b_3 \in \mathbb{R}^{64}$$

$$h_3 = \text{Relu}(z_3)$$

### **Output Layer — Dense (C, SoftMax)**

$$z_4 = W_4 h_3 + b_4 \quad W_4 \in \mathbb{R}^{c \times 64}, b_4 \in \mathbb{R}^c$$

$$\hat{y}_i = \text{SoftMax}(z_4)_i = \exp(z_{4i}) / \sum_{j=1}^c \exp(z_{4j})$$

where C = total number of word classes.

### **Loss Function — Categorical Cross Entropy**

$$L = - \sum_{i=1}^c y_i \log(\hat{y}_i)$$

### **5.6.3 Word Recognition Model**

Both models use the following training strategies to enable reliable convergence and generalization:

- Weight initialization: All dense layers are initialized with Keras default (Glorot uniform)
- We normalize the input using Standard Scaler instead of Batch Normalization, which has similar benefits with less computation overhead.
- Early training does not lead to early stopping, the number of epochs is empirically determined by looking at the stability of the validation losses.
- Model serialization: The model is serialized to a Keras H5 file (alphabet\_model.h5 or word\_model.h5) after training. It is loaded on backend startup for low latency inference.

## **5.7 Sign to Text Conversion**

During the inference stage, the backend recognition pipeline performs the following operations in order:

1. The Capture Service reads in the latest Open Pose compatible JSON file written by the Media Pipe capture loop. The JSON below contains the x and y coordinates for the 21 right hand landmarks.

2. The Recognition Service receives the JSON and extracts the 42 element feature vector and uses the pre-fitted Standard Scaler and sends the scaled vector to the loaded Keres model.
3. The model does a forward pass and returns a vector of probabilities of size 37 (or Num classes) . The predicted class is the argmax of this vector.
4. The Label Encoder decodes the predicted class index to its corresponding gesture label string (i.e. “Alif”, “Bay”, “Pay”).
5. The label is returned to the frontend as JSON response via the /api/match REST endpoint with the predicted label and recognition mode.

The Detected Letter pane displays the letter that was detected in the frontend. This prevents accidental duplication of the same letter twice in a row if the user holds the gesture (The sentence builder logic does not accept the same letter twice in a row; implemented with a use Last Letter Ref React pattern).

## 5.8 System Integration

All modules combined in one complete full stack complete web application. The Fast API backend coordinates the capture lifecycle, gesture inference, speech synthesis, and MJPEG streaming with four decoupled router modules. The detection interface, sentence builder, Learn Signs tutorial and Dictionary browser are implemented on the Next.js frontend. The frontend and backend communicate with each other via a local REST API on port 8000 with a CORS middleware to allow requests from the development frontend on port 3000.

The MJPEG stream endpoint (/api/stream) is a live video stream annotated to the frontend as a regular html image element with no need for web RTC or socket libraries client side. Media Pipe landmarks are plotted on user’s hands and body in every frame to provide visual feedback that helps users to understand the detection state of the system and correct their hand position when necessary.

The system is configurable via a pydantic-settings configuration class that takes all the runtime parameters (camera index, model paths, scaler paths, label encoder paths) from a .env file. The design allows the same code base to be deployed with different models and hardware configurations without changes to the code.

# **CHAPTER 6**

## **IMPLEMENTATION AND RESULTS**

## 6.1 IMPLEMENTATION AND RESULTS

This chapter will cover the complete implementation of the Lingua Sign system, including the technology stack and system architecture, backend and frontend realization, REST API design, real-time data pipeline, model training results, system interface design, and advanced features. This chapter depicts the evolution of Lingua Sign from a concept design to a complete, deployable full-stack application.

### 6.2 Implementation Overview

The Lingua Sign system is implemented as two components that communicate with each other via a local REST API. The trained MLP models are stored in the backend of Python Fast API which also controls the Media Pipe capture loop, performs gesture inference and synthesizes Urdu speech audio. The web interface offered to the user is implemented in Next.js 15, written in TypeScript and styled with Tailwind CSS, and polls the backend for live predictions, handles the state of the sentence builder and plays back synthesized audio.

### 6.3 Technology Stack

Table 6.3 shows the technologies used in each layer of the implemented system

| Layers            | Technology                                                                               |
|-------------------|------------------------------------------------------------------------------------------|
| Frontend          | Next.js 15, TypeScript, React 18 Hooks, CSS                                              |
| Backend           | Fast API, Uvicorn ASGI, Python 3.9+, pydantic-settings, python-dotenv                    |
| Machine Learning  | TensorFlow 2.x / Keras (Sequential Dense Networks) scikit-learn                          |
| Computer Vision   | Media Pipe Holistic (pose + 2 hands), OpenCV 4.x                                         |
| Data Storage      | SQLite 3 (main_dataset.db) – alphabet Dataset, word Dataset tables                       |
| Speech Synthesis  | edge-tts (Microsoft Edge Neural TTS, ur-PK-Uzma Neural), py game mixer, pyttsx3 fallback |
| Model Persistence | Keras h5 (.h5), scikit-learn pickle (.pkl) for scaler and encoder                        |

Table 6.3 : Lingua Sign Technology Stack

### 6.4 Backend Implementation

The backend is organized in clearly separated modules to keep decoupling between capture, recognition, speech and streaming concerns:

- main.py: bootstrap Fast API app, Setup CORS, Register routers.
- core/config.py: pydantic-settings configuration loaded from .env file  
(CAMERA\_INDEX, MODEL\_PATH, SCALER\_PATH, LABEL\_ENCODER\_PATH)

and word-model counterparts).

- routers/: The files capture.py, recognition.py, speech.py, and stream.py expose the public REST and MJPEG streaming endpoints.
- services/: mediapipe\_service.py controls capture lifecycle; recognition\_service.py does inference on latest key point Json. speech\_service.py plays cached Urdu tts audio.
- PSL/: Contains alphabet recognition and word recognition packages with training scripts, prediction modules and helper utilities for normalization, scaling, rotation and Media Pipe processing.
- data/ : models/ (trained H5 and PKL artefacts), db/main\_dataset.db (training dataset), learn/ (PSL reference images), tts\_cache/ (cached synthesized MP3 audio)

## 6.5 Frontend Implementation

Frontend is built using Next.js App Router with pages and shared components. React Hooks: use State, use Ref, use Effect for state management A set Interval polling loop makes POST requests to /api/match with a configurable cooldown interval. Main pages and components are:

- app/page.tsx: Landing page with Chroma Key hero video system demo.
- app/sign/page.tsx: Realtime detection page with camera stream + composer for letters/words/sentence + controls.
- app/learn/page.tsx: Learn Signs tutorial using PSL reference images found in data/learn/.
- app/dictionary/page.tsx: Browser able dictionary of available PSL sign categories and availability status.
- app/about/page.tsx app/contact/page.tsx app/feedback/page.tsx: Static informational pages
- components/: Shared UI components like Button, Chromakey Video, Theme Toggle and Theme Provider for dark/light theming.

## 6.6 Rest Api Endpoint

The backend exposes the following REST API endpoints that are consumed by the frontend during the detection lifecycle:

| Method | Endpoint | Description |
|--------|----------|-------------|
|--------|----------|-------------|

|      |                           |                                                                              |
|------|---------------------------|------------------------------------------------------------------------------|
| POST | /api/start-capture        | Open the webcam and begin Media Pipe capture in a background thread.         |
| POST | /api/stop-capture         | Stop the capture thread and release the webcam.                              |
| POST | /api/match                | Run inference on the latest keypoint JSON; returns predicted label and mode. |
| GET  | /api/stream               | MJPEG live video stream with Media Pipe landmarks drawn on each frame.       |
| POST | /api/speech/preload/{txt} | Pre-generate and cache the Urdu TTS audio for the given text.                |
| POST | /api/speech/{txt}         | Generate (if needed) and play the Urdu TTS audio for the given text.         |

Table 6.6 : Lingua Sign Rest Api Endpoints

## 6.7 Real Time Data Pipeline

The real time pipeline runs continuously while the user is in the Sign Detection page. OpenCV pulls frames from the configured camera device Media Pipe Holistic and pose are run on each frame and return left-hand and right-hand landmark sets. These are converted to Open Pose compatible JSON files that are written to the Key points directory at around 20 frames per second.

The recognition service for each frontend poll loads the latest JSON file, extracts the 42 element right hand feature vector, scales it with the Standard Scaler and performs a forward pass with the loaded Keras model. The predicted class index is decoded by the Label Encoder and returned back to the frontend as the recognized gesture label. The full inference loop (reading the JSON and returning a label) is under 100ms on a typical laptop CPU, and the 1 sec cooldown of the frontend is the practical bottleneck on end-to-end perceived latency.

## 6.8 Model Training and Evaluation

This section presents the design details, training configuration and quantitative evaluation results for both recognition models.

### 6.8.1 Dataset Preparation

Training data is in SQLite DB (data/db/main\_dataset.db). Each row contains 21 pairs of coordinates of the right hand landmarks and a class label . The training pipeline reads all rows, label encodes them with Label Encoder, one hot encodes them using to categorical, and applies Standard Scaler. The word dataset is divided into 80/20 train-test splits and stratified.

### 6.8.2 Training Configuration

| Hyperparameter       | Alphabet Model                                                   | Word Model                                                                   |
|----------------------|------------------------------------------------------------------|------------------------------------------------------------------------------|
| Input Features       | 42                                                               | 42                                                                           |
| Network Architecture | Dense(120) → Dropout(0.3) → Dense(64) → Dropout(0.3) → Dense(37) | Dense(256) → Dropout(0.3) → Dense(128) → Dropout(0.3) → Dense(64) → Dense(N) |
| Epochs               | 25                                                               | 50                                                                           |
| Batch Size           | 1                                                                | 32                                                                           |
| Optimizer            | Adam                                                             | Adam                                                                         |
| Loss Function        | Categorical Cross-Entropy                                        | Categorical Cross-Entropy                                                    |
| Validation Split     | 20% of training set                                              | Hold-out test set (20%)                                                      |
| Dropout Rate         | 0.3                                                              | 0.3                                                                          |
| Output Classes       | 37 (PSL alphabet)                                                | Variable (word vocabulary)                                                   |

Table 6.8.2 Training hyper-parameters for alphabet and word recognition models

### 6.8.3 Training and Testing Accuracy Results

Figure 6.8.3 Accuracy of training and testing of alphabet recognition model for 25 epochs. The training accuracy starts at about 75% in the first epoch and slowly increases to about 96% in the last epoch. During training run, test accuracy is always higher than training accuracy and around 98% at epoch 14 and remains stable in 97-99% range afterwards.

This odd discrepancy between test accuracy and training accuracy is a textbook example of dropout regularization. Dropout is a technique which randomly turns off the neurons in training, so it artificially reduces the training accuracy, but then in evaluation time the whole network is used and gives a higher test accuracy.

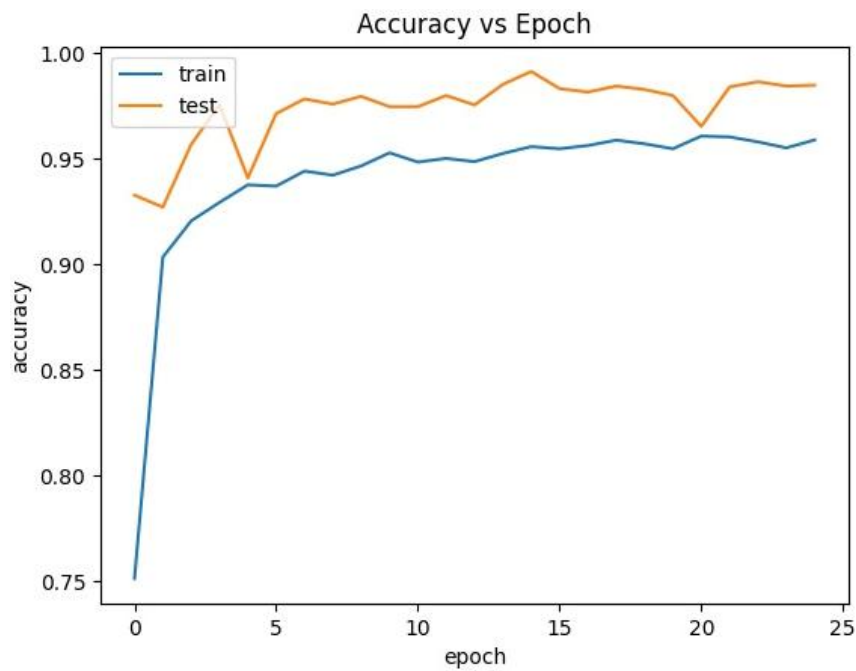


Figure 6.8.3 shows the training and testing accuracy

### 6.8.4 Confusion Matrix Analysis

Figure 6.8.4 shows the normalized confusion matrix for the alphabet model on the held-out test set. The matrix is row normalized so that each entry represents the fraction of true instances of a given class predicted to be another class. A perfect classifier produces a diagonal of 1.00 values only.

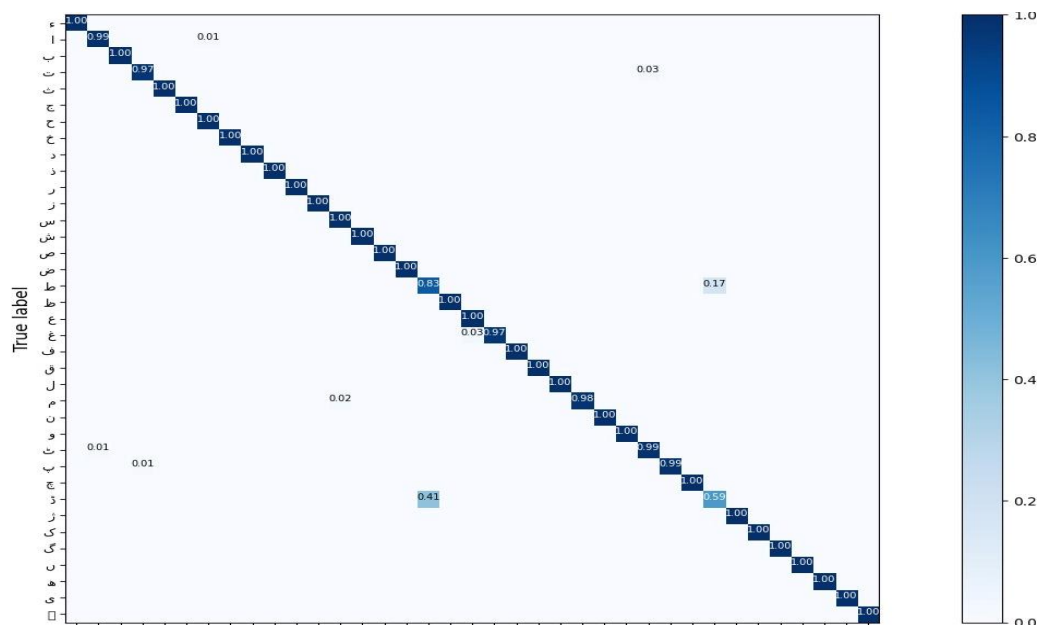


Figure 6.8.4 shows the confusion matrix for alphabet model

From the confusion matrix, it can be seen that the majority of the 37 classes of PSL alphabets are recognized with 1.00 accuracy on the test set, which shows excellent class separability for most of the gestures. Some borderline training samples confuse some classes in the range of 0.01-0.03. 0.83 correctly identifies one class but in 0.17 it is misclassified to a visually similar letter. There are two big confusions, one of which is only misclassified 0.59 and 0.41. These two challenging classes are PSL characters with very similar 2D hand shapes in Media Pipe landmark space and are prime candidates for targeted dataset expansion.

### 6.8.5 Performance Metrics Summary

| Metric                                     | Value                                          |
|--------------------------------------------|------------------------------------------------|
| Number of PSL Alphabet Classes             | 37                                             |
| Final Training Accuracy (Alphabet Model)   | $\approx 96\%$                                 |
| Final Test Accuracy (Alphabet Model)       | $\approx 98.28\%$                              |
| Mean Per-Class Diagonal (Confusion Matrix) | $\approx 0.98$                                 |
| Media Pipe Capture Rate                    | $\approx 30$ FPS                               |
| End-to-End Prediction Latency              | $\approx 1$ second (frontend cooldown limited) |
| TTS Audio Latency (cached)                 | $< 0.5$ seconds                                |
| TTS Audio Latency (first synthesis)        | 2-4 seconds (neural TTS generation)            |

Table 6.8.5 : Performance Metrics Summary

Overall, the alphabet model is well above the greater than 90% accuracy target specified in the Non-Functional Requirements. The clean confusion matrix and high test accuracy demonstrate that the combination of landmark-based features and a compact MLP architecture is well suited to the PSL alphabet classification task.

## 6.9 System Interface and Output Design

The interface of Lingua Sign is a modern dark-themed web application built with Next.js 15 and Tailwind CSS. The application can be launched from the local development URL in any modern browser and provides the user four main screens.

### 6.9.1 Landing Page

The two column hero on Landing Page introduces the system. In the left column we have the project headline, short description and two main call to action buttons (Start Detection and Learn Signs). 37 Letters 6 Words Real Time Detection Urdu Speech Output Here is a line of statistics summing up system capabilities. Right column: ambient branding by way of a

Chroma Key video of a signing character. It has a light/dark mode switch. Landing Page: 6.9.1. Fig.

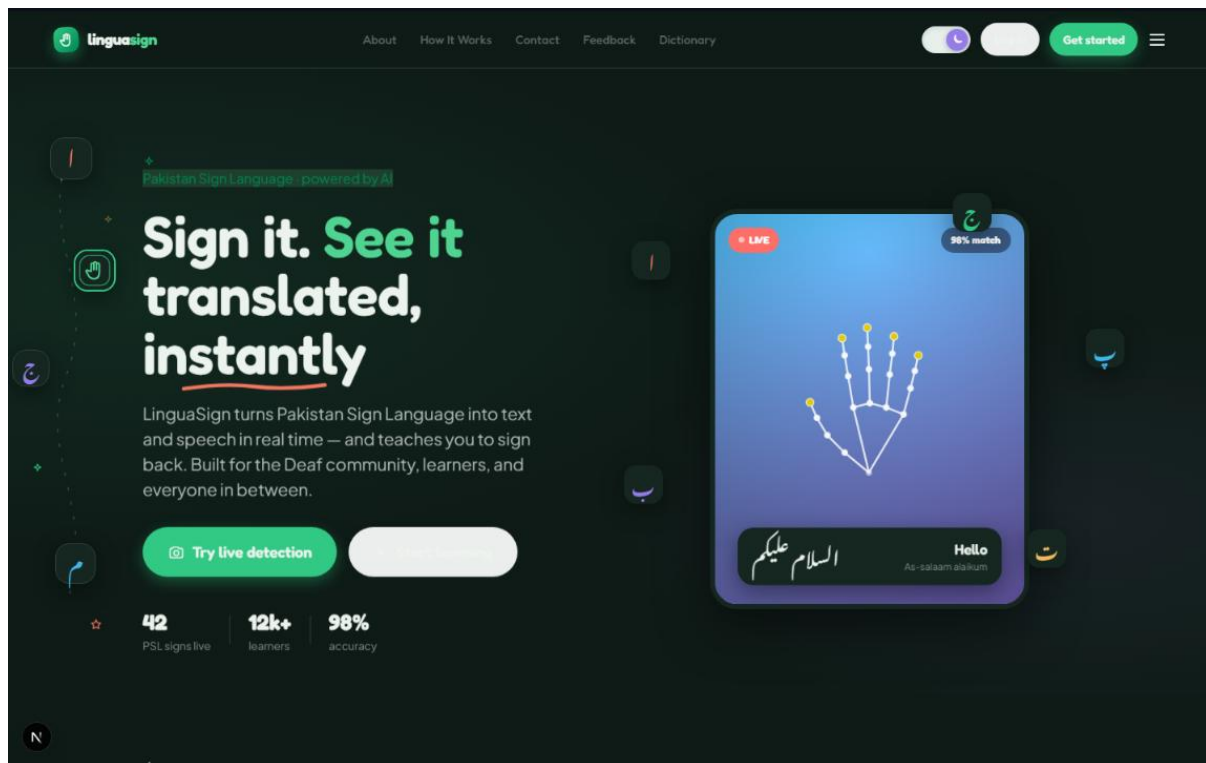


Figure 6.9.1: Landing Page

## 6.9.2 Sign Detection Page

Detection of page signs This is the main page for operation. The left control panel shows a LIVE status indicator, switches for Word Mode and Speech, and three stacked output cards: Detected Letter, Current Word, and Sentence. Each card has an action button (Accept Letter, Add Word to Sentence, Speak Sentence). The sentence card has copy to clipboard, clear and click to remove individual words. On the right panel you can see the live MJPEG camera stream with the Media Pipe landmarks overlaid. Below the feed there is a large button to switch between Start Detection and Stop Detection. The Sign Detection Page is shown in Figure 6.9.2

On the right panel is a live feed of the MJPEG camera with Media Pipe hand, pose and face landmarks overlaid in real-time. This allows users to check that their gestures are tracked correctly. There is a large button under the feed making it easy to toggle between Start Detection and Stop Detection to control the detection process. At runtime, predictions are updated in near real-time with low latency so that the user experience is interactive and responsive. The page is designed in a clean and intuitive way with minimal effort from the user

and at the same time all the essential controls and outputs are easily accessible. Figure 6.9.2  
Screen for Detecting Signs

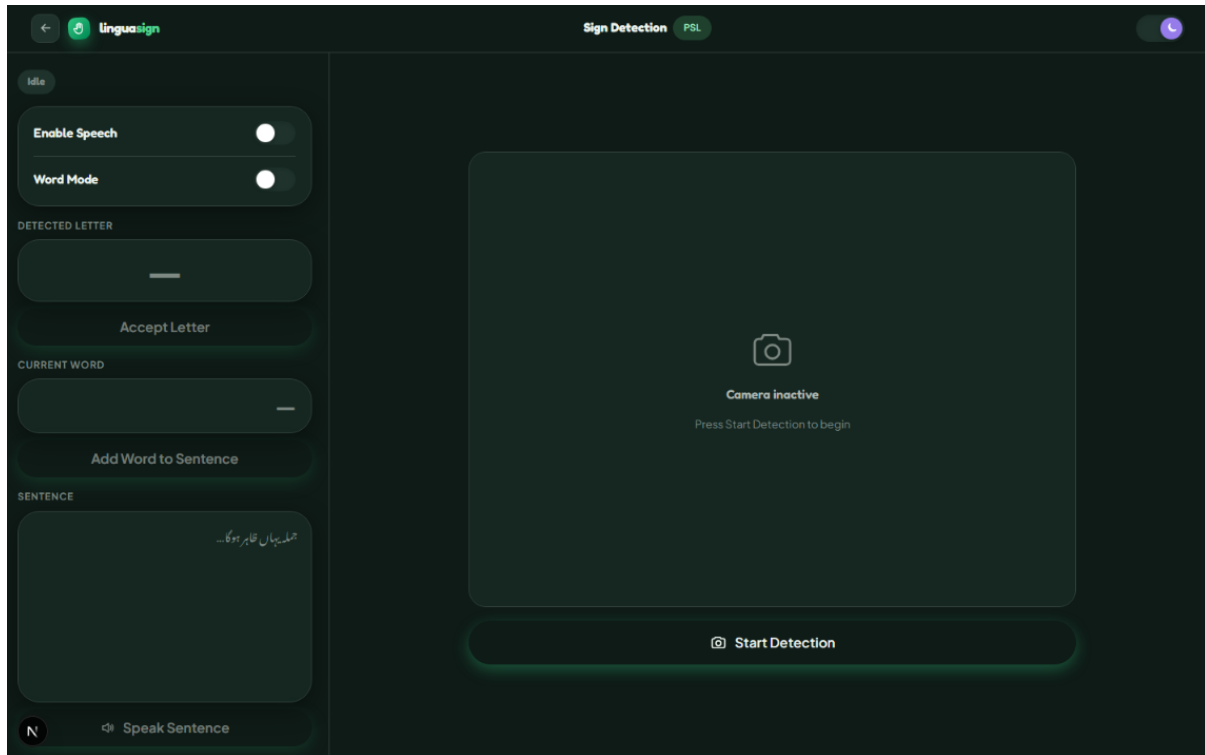


Figure 6.9.2 Detecting page

### 6.9.3 Learn Sign Page

Learn Signs Page has an interactive tutorial and practice tool. A tab control allows the Learn Mode (user walks through the PSL alphabet at own pace with Previous and Next controls) to be changed to Quiz Mode (random target letters are presented with a running score). The left panel displays the reference image and the current target letter, and the progress through the alphabet; The right panel shows a live camera feed and feedback card that indicates whether the system has detected if yes the system with a green Correct badge or if not red Wrong badge . 6.9.3 . Page | Learn Signs

The left panel displays the reference picture of the chosen sign, the current target sign and the learner's progress through the alphabet, offering a clear visual orientation throughout the session. The right panel displays the live camera feed, with real-time hand tracking, and the feedback card showing the letter detected by the system. And then, at the user's gesture, if the detected sign is the same as the target letter, the system shows a green Correct badge, otherwise a red Wrong badge. Users can get instant feedback to find their errors and improve their signing

accuracy. The page is designed to be an engaging, user-friendly learning experience with guided tutorials, interactive quizzes, and AI-powered instant feedback.

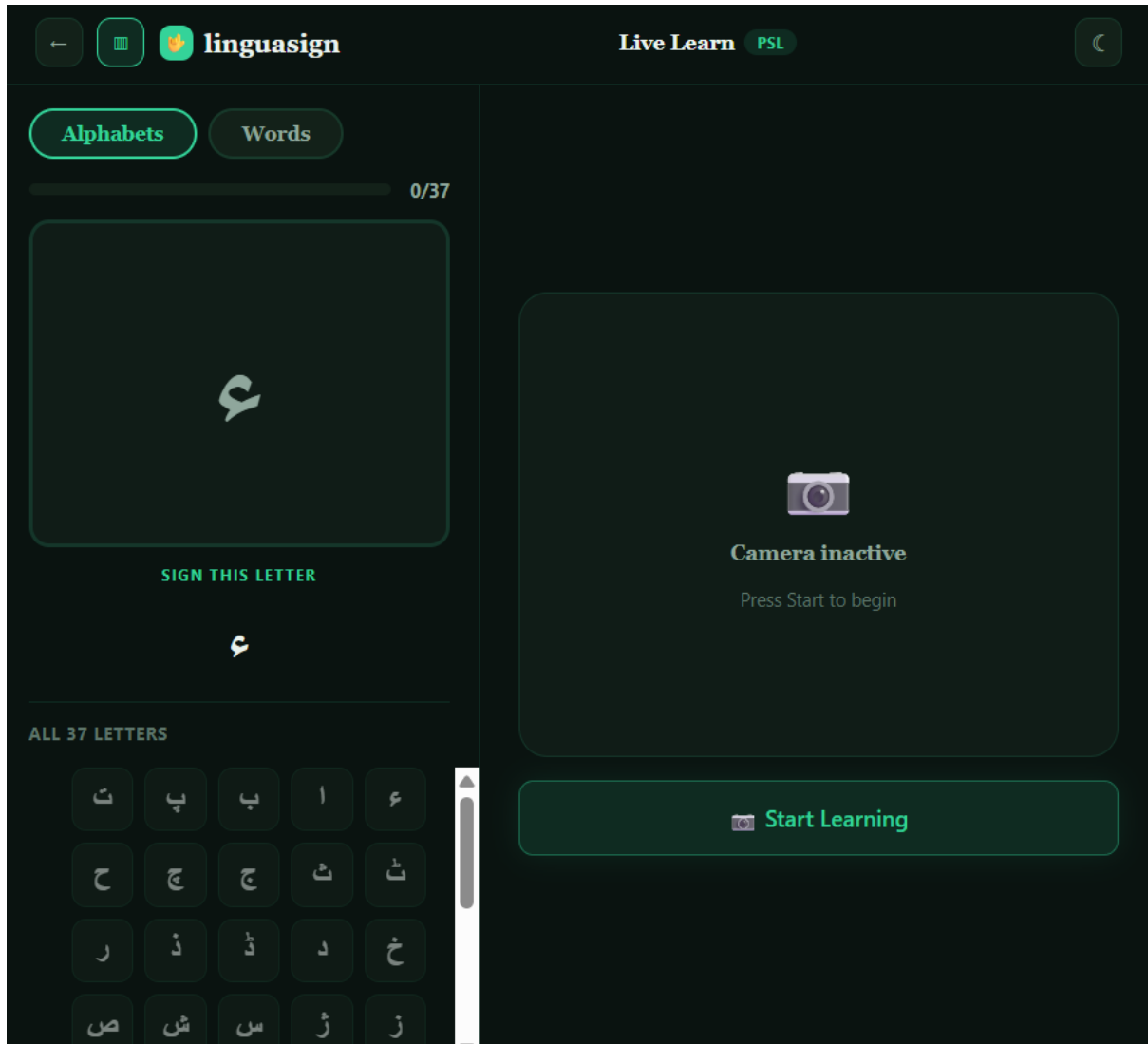


Figure 6.9.3 Sign Learn Page

#### 6.9.4 Dictionary Page

You can use the Dictionary Page to browse PSL signs by category. Each category has a card with an icon, title, subtitle and a status pill with either Ready (with the number of signs) or Coming soon. The current release contains the 37 letter Urdu Alphabet and 5 word .Words category with plans to expand to include Greetings, Numbers, Colors, Family, Common Phrases, Animals and Food in the future Figure 6.9.4 : The Dictionary Page.

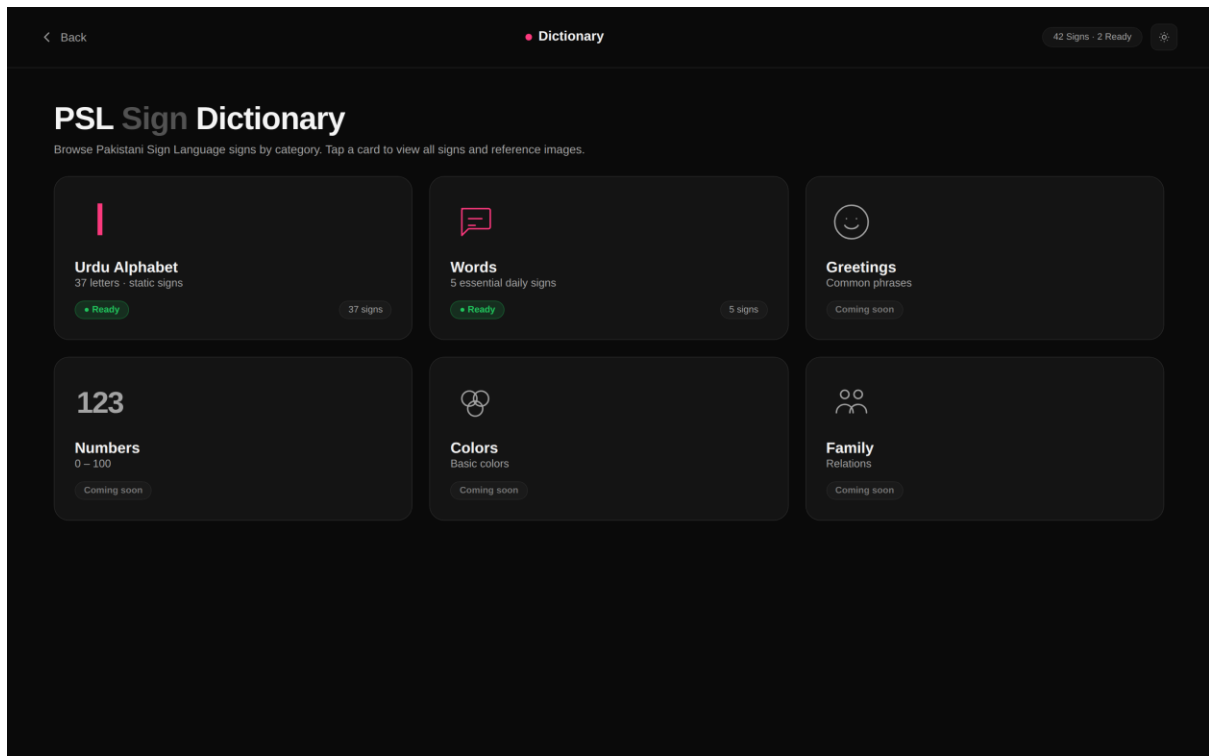


Figure 6.9.4 : The Dictionary Page.

### 6.9.5 Dashboard Page

The dashboard stores the user's activity in the browser's local storage, so that all learning records are available after restarting the application. All data is stored locally, no need for cloud storage and privacy protected. The visual display of the learning statistics motivates the users to practice regularly and enables a more engaging learning experience similar to modern educational apps.

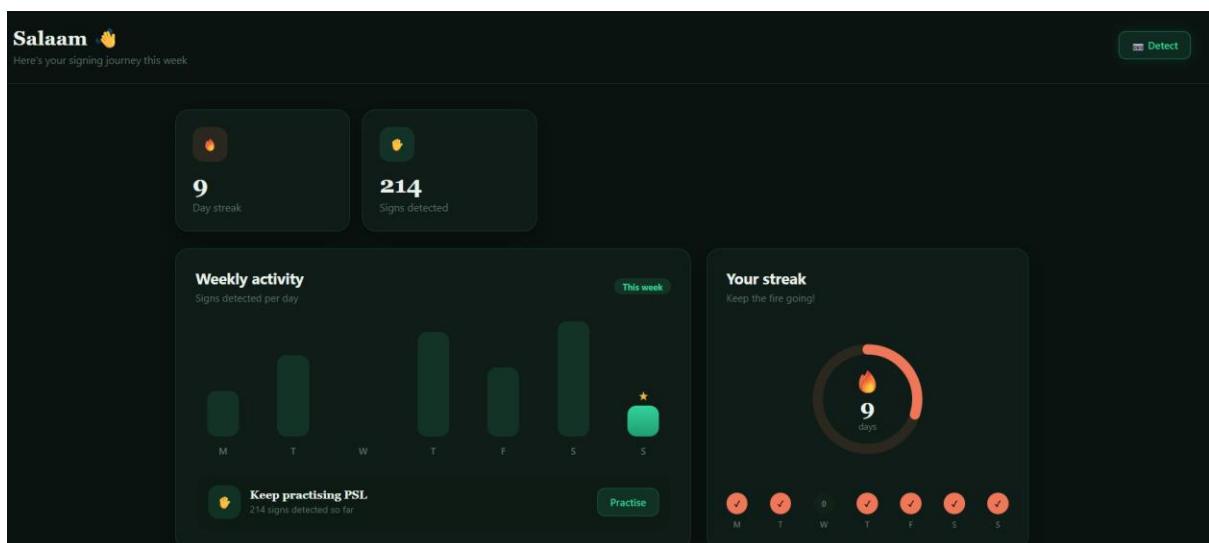


Figure 6.9.5 : Dash Board Page.

### 6.9.6 Setting Page

All configuration settings are stored locally in the browser, in Local Storage. So if the application is reopened and the user has no online account or cloud synchronization, the preferences are restored automatically. The Settings page improves usability by allowing the user to configure the interface while the system is offline.

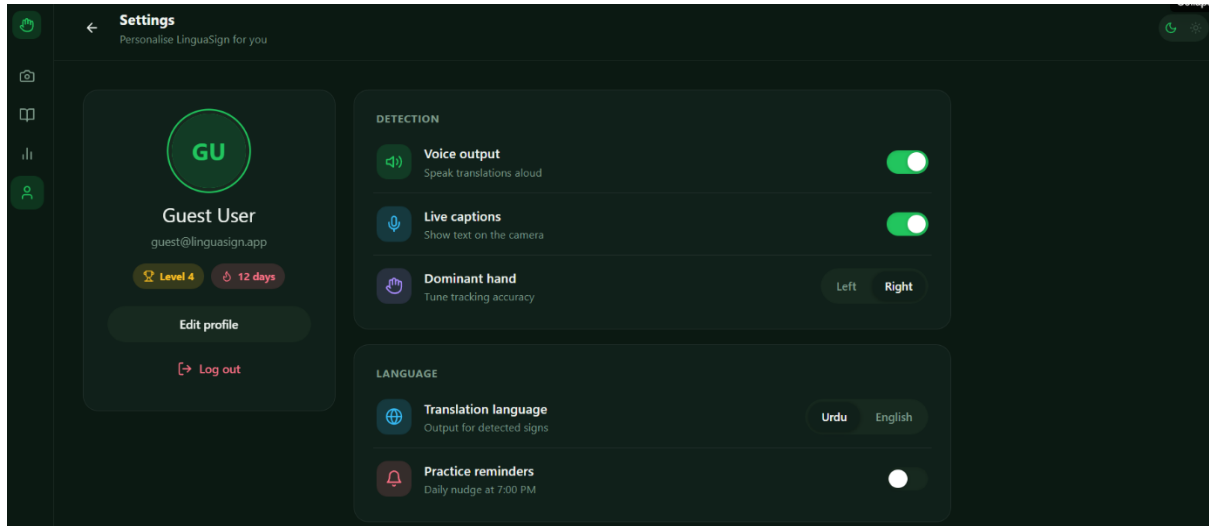


Figure 6.9.6 : Setting Page.

## 6.10 Advanced System Features

In addition to the core recognition pipeline, many advanced features have been implemented to adapt Lingua Sign for practical, day-to-day communication.

### 6.10.1 Sentence Builder

The biggest step up from a simple one letter classifier is the sentence builder. Detection is done on three levels of hierarchy: current recognized letter, current composed word and the whole sentence. The user accepts a detected letter by pressing Accept Letter, which adds the letter to the current word. Click Add Word to Sentence to send the entire word to the sentence list. Words can be clicked on to delete them, the whole sentence can be copied to the clipboard or spoken out loud.

### 6.10.2 Dual-Mode Recognition

Lingua Sign comes with two independently trained models: a 37 class alphabet model and a word model trained on full gesture words. They are toggled by a Word Mode toggle in the UI that toggles the mode field in each /api/match request. The backend recognition service routes

inference to the appropriate model without interrupting the Media Pipe capture stream, enabling the user to switch between letter-level and word-level recognition at runtime.

### **6.10.3 Urdu Text to Speech**

Speech output generated by Microsoft Edge Neural TTS with voice ur-PK-Uzma Neural. The system pre-generates and caches MP3 audio files, indexed by the MD5 hash of the text. On the first request, edge-tts will generate the audio asynchronously, but subsequent requests will play the cached file through the pygame mixer with sub-second latency. If the backend TTS service is not available, it falls back to the browser Web Speech API as a client-side TTS service.

### **6.10.4 Real Time MJPEG Video Streaming**

The `/api/stream` endpoint serves a MJPEG stream with Media Pipe landmarks annotated on each frame. By showing these landmarks, users can know the state of hand detection of the system and correct the signing posture when the gestures are not recognized correctly.

### **6.10.5 Configurable Backend and Re Training Pipeline**

All runtime parameters are loaded from a `.env` configuration file using `pydantic-settings` so it can be deployed on different camera devices, model versions and directory structures without code changes. Two capture scripts (`capture_alphabets.py` and `capture_words.py`) will collect new landmark samples to insert into the SQLite database and `retrain.py` will do a full re-training pass against the updated tables. Helper utilities in `PSL/helper/` are used to perform geometric augmentation and data synthesis to augment the continuous dataset.

### **6.10.6 Learning Analytics Dashboard**

The dashboard gives users measurable indicators of their performance, unlike traditional learning applications which just display educational content this allows learners to identify their strengths and weaknesses and motivates them to practice consistently through a streak-based reward system. All analytics are created locally with browser storage, your privacy is protected and no personal data is sent to other servers.

# **CHAPTER 7**

## **CONCLUSION AND FUTURE WORK**

## 7.1 CONCLUSION

In this project, an AI based real-time Pakistani Sign Language recognition and translation system, Lingua Sign, has been designed, developed and evaluated. It uses Media Pipe Holistic for hand landmarks, a small Multilayer Perceptron neural network for gesture classification and Microsoft Edge Neural TTS for Urdu speech synthesis. Lingua Sign is a full-stack web app, with a Python Fast API backend and a Next.js 15 frontend. It shows highly accurate and practical camera-based sign language recognition without any special hardware or internet connection.

Our Project's main achievements:

- PSL dataset was developed comprising 37 alphabet classes and a set of common word gestures, collected from multiple signers in a variety of lighting and environmental conditions.
- The alphabet recognition model had a test accuracy of ~98.28% (well above the non-functional accuracy requirement of 90%) and was running at ~30 FPS with an end-to-end latency of ~1 second.
- Developed and deployed a full client-server web application with complete recognition pipeline, sentence builder, dual mode recognition (Letter and word), real time MJPEG video streaming and Urdu neural TTS output.
- Fully offline system to be used in low resource settings and for privacy protection by processing all data locally.

The analysis of the confusion matrix shows the recognition rate of two classes of the PSL alphabet is less than 0.80. This is due to the high visual similarity of the two classes in the two-dimensional landmark space. These examples give a clear direction for further expansion of the dataset and improvement of the model.

Lingua Sign use Media Pipe technology by addressing a real and critical communication barrier for Pakistan's deaf and speech-impaired community. It provides a good starting point for further development into a production ready assistive communication tool with focus on usability, regional language support and offline capability.

## **7.2 Future Work**

The current implementation demonstrates a working and correct real-time PSL recognition system, but there are many possibilities to extend its capabilities, robustness and applicability in the real world. The following directions for future work are suggested:

### **7.2.1 Production Deployment**

At the moment, it is realized as a local web application and is intended for research and demonstration purposes. Future work will include:

- Implementation of the system as a cross-platform mobile application (Android and iOS) to increase the accessibility for deaf users in their daily lives.
- Deploying backend on cloud server with global content delivery network for remote access with optional offline operation by locally cached model.
- Optimize MLP model for edge deployment with TensorFlow Lite quantization or ONNX conversion for inference on mobile CPU without loss of performance

### **7.2.2 Dynamic Gesture and Sentence-Level recognition**

The current system classifies single frames with single static gestures. For the extension to continuous sign sentence recognition:

- Feeding temporal sequences of landmark vectors (sliding windows of 30-60 frames) into LSTM or Transformer-based sequence models
- Use facial expression and body posture landmarks together as complementary features for gestures that include non-manual markers.
- Segmentation mechanism for automatic detection of gesture boundaries in continuous signing streams without explicit user interaction.

### **7.2.3 Dataset Expansion**

The current dataset of the PSL has limited vocabulary and signer diversity. Future development of datasets should include:

- Increased vocabulary with common conversational phrases, numbers, colors, family relations, and domain-specific terminology for healthcare and education contexts.
- To draw a larger and more diverse group of signers including signers of different ages, genders and signing proficiencies.

- Collaborate with the deaf community and PSL institutions in Pakistan to collect naturalistic signing data and to validate the completeness of the gesture vocabulary.

#### **7.2.4 Intelligent Analytics and Adaptive Learning**

Future versions of LinguaSign could include:

- AI-based performance monitoring to analyze patterns of recognition accuracy for different users and conditions.
- Adaptive learning mechanisms that detect stable misclassification patterns and invoke targeted data collection or fine-tuning passes.
- Usage analytics to determine the most-used gestures, and to prioritize vocabulary expansion accordingly.

#### **7.2.5 Domain Expansion**

LinguaSign's modular architecture can be extended to other application domains:

- Healthcare: Improving communication between deaf patients and healthcare professionals in the clinical encounter.
- Education: Creation of inclusive classrooms and e-learning platforms for students with hearing impairment.
- Public Services: Enable better communication in banks, government offices and emergency services.
- Multilingual support: The system can be extended to other regional sign languages such as Afghan Sign Language and British Sign Language with minimal architectural changes.

# REFERENCES

- [1] United Nations, International Day of Sign Languages. United Nations, 2024.
- [2] N. A. Nasharudin, A. A. Shafie, and S. Hussain, "YOLO-based deep neural network for American Sign Language alphabet recognition," *Journal of Intelligent & Fuzzy Systems*, vol. 41, no. 2, pp. 2699–2711, 2021.
- [3] R. Kumari and A. Anand, "Hybrid CNN–LSTM model for sign language recognition using the WLASL dataset," *Multimedia Tools and Applications*, vol. 81, no. 19, pp. 26945–26962, 2022.
- [4] M. Jawas, M. Al-Obaidy, and R. Al-Obaidy, "Vision-based sign language recognition using RGB video and optical flow," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 6, pp. 141–150, 2022.
- [5] R. Kumari and A. Anand, "Hybrid CNN-LSTM with attention mechanism for isolated word recognition from WLASL dataset," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 5, pp. 112–120, 2021.
- [6] M. El Rwelli, A. T. Mahmoud, and H. M. Ahmed, "Glove-based CNN system for Arabic Sign Language recognition and voice translation," *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, pp. 1234–1245, 2020.
- [7] Z. Halim and G. Abbas, "A Kinect-based sign language hand gesture recognition system for hearing- and speech-impaired: A pilot study of Pakistani Sign Language," *Assistive Technology*, vol. 27, no. 1, pp. 34–43, 2015.
- [8] H. M. Hamza and A. Wali, "Pakistan Sign Language recognition: Leveraging deep learning models with a limited dataset," *Machine Vision and Applications*, vol. 34, no. 5, p. 71, 2023.
- [9] M. Imran, A. Razzaq, I. A. Baig, A. Hussain, S. Shahid, and T. Rehman ur, "Dataset of Pakistan Sign Language and automatic recognition of hand configuration of Urdu alphabet through machine learning," *Data in Brief*, vol. 36, 2021.
- [10] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [11] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv:1906.08172*, 2019.
- [12] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: <https://www.tensorflow.org/>
- [13] S. Ramírez, FastAPI: Modern, fast (high-performance) web framework for building APIs with Python, 2018–2025. [Online]. Available: <https://fastapi.tiangolo.com/>

- [14] Vercel, Next.js — The React Framework for the Web, 2024–2025. [Online]. Available: <https://nextjs.org/>
- [15] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. 3rd International Conference on Learning Representations (ICLR), San Diego, CA, 2015.
- [16] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [17] Microsoft Corporation, "Microsoft Edge TTS — ur-PK-UzmaNeural Voice," Azure Cognitive Services, 2023. [Online]. Available: <https://azure.microsoft.com/en-us/products/cognitive-services/text-to-speech>